SONIC: Supersizing Motion Tracking for Natural Humanoid Whole-Body Control

Zhengyi Luo[†], **Ye Yuan**[†], **Tingwu Wang**[†], **Chenran Li**[†], **Fernando Castañeda**[†], **Sirui Chen**[*], **Zi-Ang Cao**[*], **Jiefeng Li**[*], **David Minor**[*], **Qingwei Ben**[*], **Jinhyung Park**[*], **David Sami**[*], **Zi Wang**[*], **Xingye Da**[*], **Runyu Ding**, **Cyrus Hogg**, **Lina Song**, **Edy Lim**, **Eugene Jeong**, **Tairan He**, **Haoru Xue**, **Wenli Xiao**, **Simon Yuen**, **Jan Kautz**, **Yan Chang**, **Umar Iqbal**, **Linxi "Jim" Fan**[‡], **Yuke Zhu**[‡]

NVIDIA, 2788 San Tomas Expressway, Santa Clara, CA, USA

[†] Co-first Authors, [*] Core Contributors, [‡] Equal Advising

https://nvlabs.github.io/GEAR-SONIC/

arXiv:2511.07820v4 [cs.RO] 13 Aug 2026

Abstract

Despite the rise of billion-parameter foundation models trained across thousands of graphical processing units (GPUs), similar scaling gains have not been shown for humanoid control. Current neural controllers for humanoids remain modest in size, target a limited set of behaviors, and are trained on a handful of GPUs. We show that scaling model capacity, data, and compute yields a generalist humanoid controller capable of natural, robust whole-body movements. We position motion tracking as a scalable task for humanoid control, leveraging dense supervision from diverse motion-capture data to acquire human motion priors without manual reward engineering. We build a foundation model for motion tracking by scaling along three axes: network size (1.2M to 42M parameters), dataset volume (100M+ frames from 700 hours of motion capture), and compute (21k GPU hours). Beyond demonstrating the benefits of scale, we further show downstream utility through a real-time kinematic planner that bridges motion tracking to tasks such as navigation, enabling natural and interactive control, as well as a unified token space that supports virtual reality (VR) teleoperation and vision-language-action (VLA) models with a single policy. Through this interface, we demonstrate autonomous VLA-driven whole-body loco-manipulation requiring coordinated hand and foot placement. Scaling motion tracking exhibits favorable properties: performance improves steadily with compute and data diversity, and learned policies generalize to unseen motions, establishing motion tracking at scale as a practical foundation for humanoid control.

1. Introduction

Over the past decade, artificial intelligence has scaled rapidly: the generative pretrained transformer (GPT) [1] family of models is trained on 25,000+ graphical processing units (GPUs) with trillions of tokens; video and image-generation models [2–6] leverage thousands of GPUs processing billions of images. These foundation models have shown a consistent pattern: scale unlocks emergent capabilities, generalization, and robustness that smaller models cannot achieve [7–9]. Yet for sim-to-real humanoid control, similar scaling gains have not been achieved. State-of-the-art humanoid control policies are often small neural networks, such as three-layer multilayer perceptrons (MLPs), trained on a few GPUs for a single task. Manually engineered reward terms are designed per task and do not generalize across behaviors, fundamentally limiting the scalability of these approaches.

Why hasn't humanoid control scaled? The fundamental issue is task selection. Tasks like locomotion require extensive reward engineering for each scenario—walking forward naturally provides little signal for dancing [10], getting up from the ground [11, 12], or teleoperation [13–15]. Each new capability demands redesigned rewards and objectives, making scaling up difficult. Generative imitation methods such as AMP [16], ASE

© 2026 NVIDIA. All rights reserved.

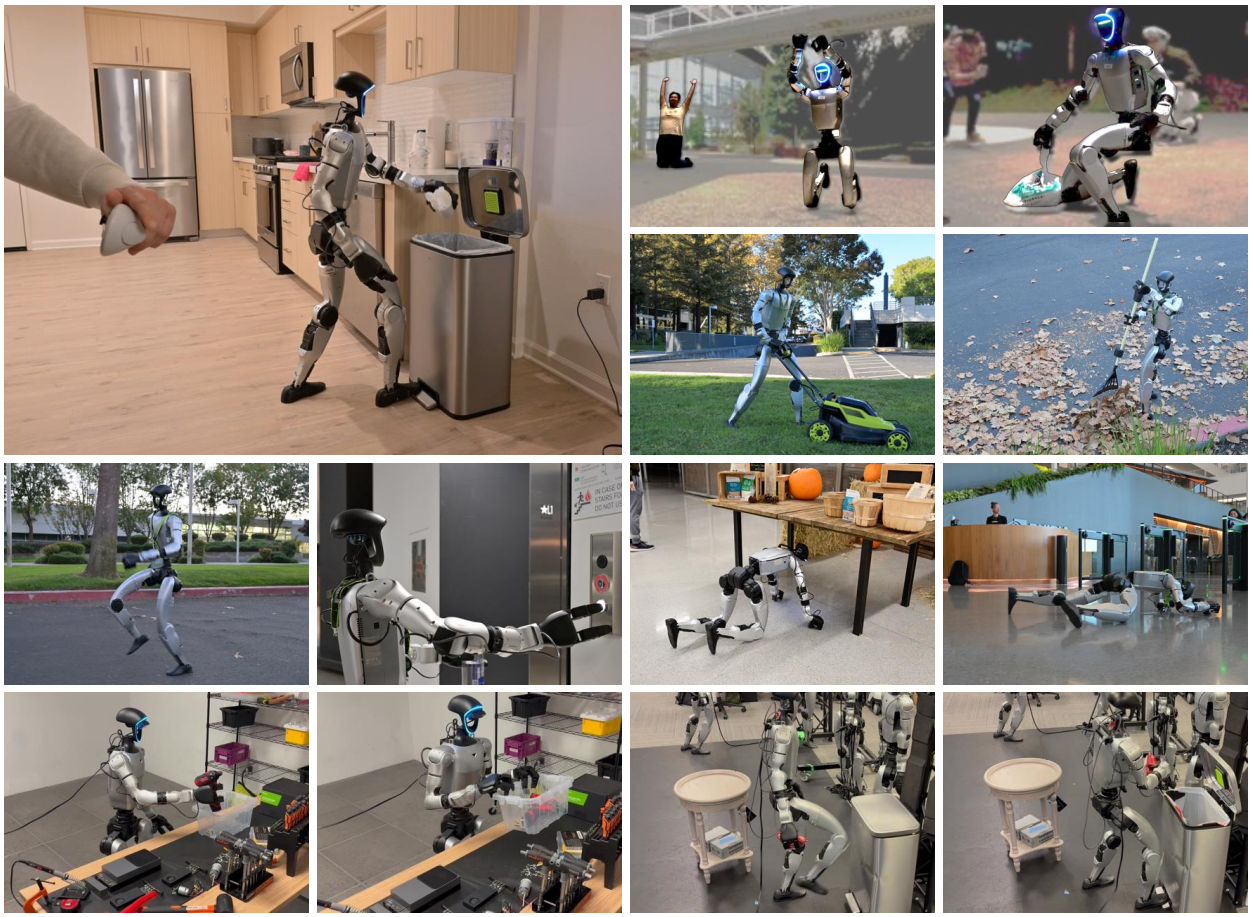

Figure 1: SONIC enables diverse humanoid tasks through a universal control policy that handles diverse input modalities and control interfaces.

[17], and CALM [18] provide a unified objective by combining matching motion distributions and simple task-specific rewards, but prior work has shown that their discriminator-based training signal is prone to mode collapse as the motion dataset grows in size and diversity [19, 20]. Even if we identify a scalable objective that can learn diverse behaviors, a second challenge emerges: how do we support the diverse range of real-world applications? A desired humanoid controller should handle teleoperation, goal-directed tasks, navigation, and even vision-language commands [21–25]. Building a system that scales while remaining flexible for different task specifications is non-trivial.

In this work, we address both challenges by identifying motion tracking as the scalable foundational task for humanoid control. Motion tracking leverages human motion capture data, which provides dense, frame-by-frame supervision without reward engineering. Critically, humanoids benefit from decades of motion capture research—datasets covering walking, running, dancing, sports, and object interactions already exist at scale [26–28]. Although there exist prior works on motion tracking [29–37], they are mostly limited to showing whole-body motion tracking results on **training data** and have not demonstrated many downstream tasks beyond motion tracking or navigation. We supersize physics-based motion tracking to 100 million frames (at 50 Hz) with 128-GPU training, achieving universal tracking capabilities across diverse human behaviors while maintaining real-time performance. In addition, we show how such a motion tracker can be applied to meaningful downstream tasks, and introduce two key contributions. First, we develop a universal kinematic motion generation system for interactive control, enabling goal-directed tasks such as interactive locomotion and game-like character control through kinematic planning in motion space. Second, we design a universal

token space that unifies heterogeneous motion sources, including VR teleoperation, vision-language-action models, and generative motion models that convert video, text, and music into motion [38–41], within a single control interface.

We propose *Supersizing mOtion tracking for Natural humanoId Control* (SONIC), a framework that enables natural humanoid control across a wide range of applications (Movie S1). We achieve high-precision teleoperation and interactive control capabilities, including running, jumping, and crawling with natural human-like movement. Leveraging our universal token space, our controller can directly map estimated whole-body human motion to humanoid control signals, bypassing the need for explicit retargeting at runtime. We integrate our tracking policy with multi-modal human motion generation models, supporting video, text, and music control. Furthermore, we show that teleoperation data collected through our system can be used to train vision-language-action foundation models [42], establishing a complete pipeline from motion tracking at scale to foundation model-based humanoid control, including tasks that demand simultaneous hand-foot coordination for whole-body loco-manipulation. These results validate that large-scale motion tracking serves as a practical foundational task for diverse real-world humanoid applications.

Our contributions are threefold. First, we identify motion tracking as a scalable foundational task for humanoid control, demonstrating that it exhibits favorable scaling properties with both compute and data diversity; we scale up humanoid control to 21k GPU hours and 100 million frames of motion sequences, achieving universal tracking capabilities across diverse human behaviors. Second, we introduce a real-time kinematic motion generator for interactive control and a universal token space with specialized encoders for robot, human, and hybrid motion inputs, all mapped into a shared quantized representation. Third, we provide a comprehensive evaluation demonstrating humanoid control scaling trends, zero-shot transfer to unseen motions, robust sim-to-real deployment on physical humanoid robots, and successful integration with foundation models; we show that the universal token space enables VLA-driven whole-body loco-manipulation, including tasks requiring coordinated hand grasping and precise foot placement, across five real-world tasks.

2. Results

We used the Unitree G1 humanoid [44] to demonstrate our supersizing humanoid motion tracking framework, SONIC. We demonstrated SONIC's motion tracking capabilities, interactive kinematic motion planning, multi-modal control, teleoperation, and loco-manipulation, shown in Fig. 1.

2.1. Motion Tracking

SONIC, trained on 100 million frames of motion over 21k GPU hours (128 GPUs over 7 days), exhibited strong generalization to **unseen** motions. In this section, we evaluated the generalization capabilities of our tracker on large-scale, unseen motion datasets in simulation and the real world.

Metrics. We employed a comprehensive set of pose-based and physics-based metrics to measure motion imitation performance. The primary measure was the success rate (Succ), where a motion imitation was deemed unsuccessful if the humanoid deviated too far from the reference motion trajectory. We further reported the local (root-relative) mean per-joint position error (MPJPE-L) E_{mpjpe} (in millimeters, mm), computed over 14 body links (pelvis, knees, ankles, torso, elbows, wrists) [29], quantifying the accuracy of the imitation in the robot's local frame. To assess physical fidelity, we also calculated differences in acceleration (E_{acc}, mm/frame^2) and velocity (E_{vel}, mm/frame) between the simulated humanoid and the reference human motion.

Evaluation Protocol and Dataset. We evaluated on three test sets (Tab. 2). From our motion-capture dataset, we constructed two held-out splits. The first, **test-content** (7,016 clips, 15 hours), evaluated generalization to *unseen motion content* and contained 182 sub-categories entirely absent from training. The second, **test-repetition** (9,395 clips, 12 hours), evaluated robustness to *new performances and repetitions* of known motion types. For baseline comparisons, we additionally evaluated on **PHUMA** [43], a publicly available dataset of

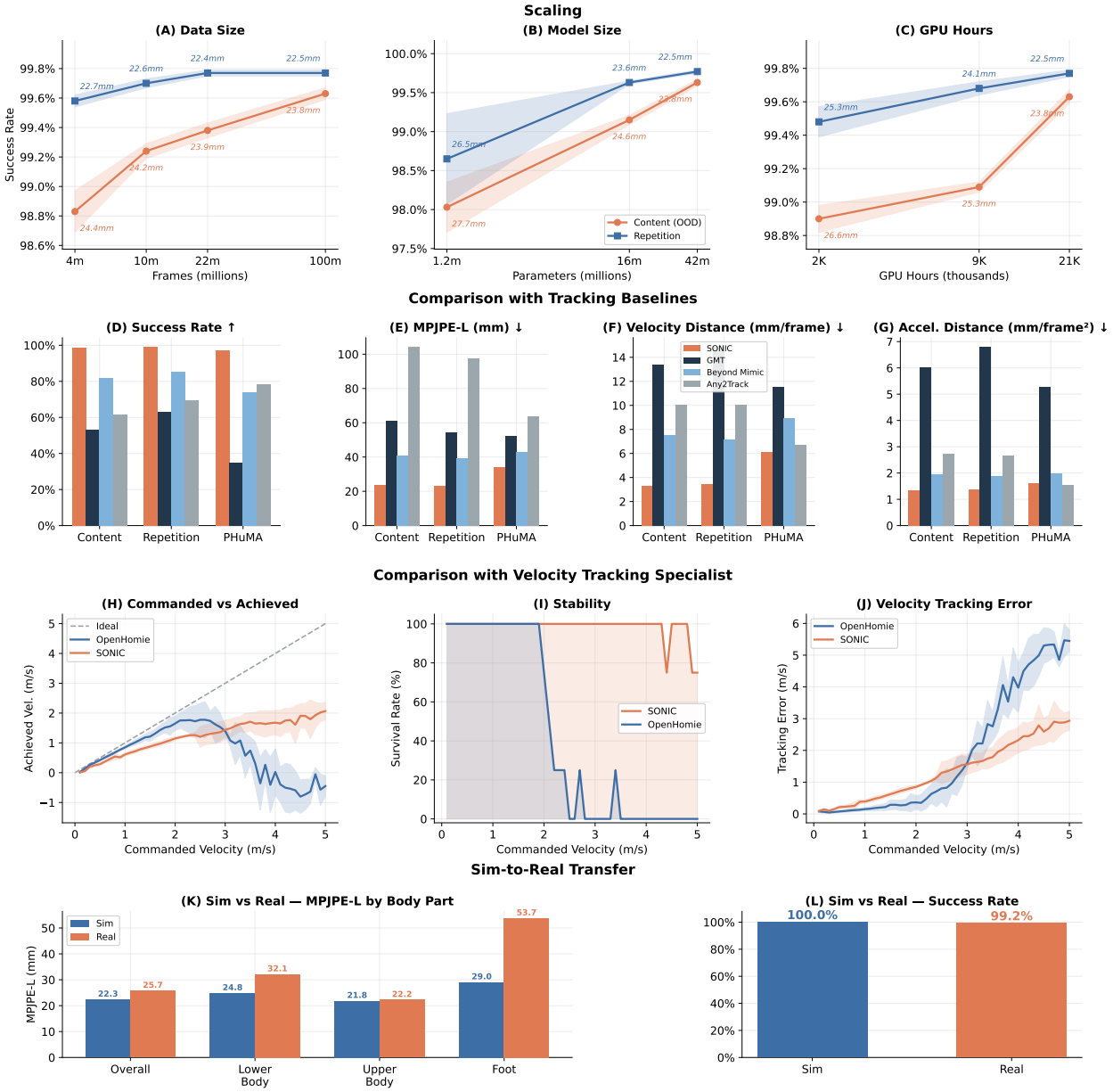

Figure 2: **Scaling and benchmarking of `SONIC` for universal motion tracking. (A to C)** Effect of scaling data size, model size, and compute on test-content (unseen motion content, OOD) and test-repetition (held-out takes of seen motion types). Lines and shaded bands: mean ± 1 s.d. over $n{=}6$ evaluations per configuration. Success rate improves along all three axes (Welch's t-test, all $P \leq 0.011$). **(D to G)** Comparison with baselines on test-content, test-repetition, and PHUMA [43] ($N{=}7{,}016$, $9{,}395$, and $68{,}326$ motions): (D) success rate (`SONIC` vs. each baseline, Welch's t-test, all $P < 0.001$); (E) MPJPE-L, (F) velocity distance, and (G) acceleration distance, reported as means over successfully tracked motions. **(H to J)** Velocity tracking vs. the OpenHomie specialist [13]: (H) commanded vs. achieved velocity, (I) survival rate ($197/200$ vs. $86/200$ runs, Welch's t-test, $P < 0.001$), and (J) tracking error (paired t-test across commanded velocities, $P < 0.001$ in both velocity regimes); mean ± 1 s.d. over $n{=}4$ directions. **(K and L)** Sim-to-real transfer: (K) MPJPE-L by body part and (L) success rate on the same $N{=}124$ motions (sim $124/124$ vs. real $123/124$).

68,326 motions from a different retargeting pipeline. Motion imitation was considered unsuccessful if the humanoid's root height or end-effector height deviated by more than 0.25 m from the reference. Unlike prior work that defines success via global root position error [34] (for example, a 0.5 m threshold), our tracker performed *local* motion tracking rather than following a global trajectory. Our metric, similar to [29], captured the physically meaningful failure modes such as falling. See the Humanoid Motion Dataset section for dataset details.

Scaling Up Motion Tracking. In Fig. 2(A–C), we analyzed scaling along three axes. For data size, we compared training on 4M, 10M, 22M, and 100M frames (corresponding to 20k, 50k, 110k, and 317k motion clips); smaller subsets were created by uniformly sampling across motion sub-categories to preserve distribution diversity. For model size, we scaled from 1.2M to 16M to 42M parameters. For compute, we trained on 2, 4, and 16 nodes (16, 32, and 128 GPUs), all to 50k iterations, yielding approximately 2k, 9k, and 21k GPU hours. All evaluations used Isaac Lab [45], with models trained 50k iterations (when performance usually plateaus). Scaling yielded consistent improvements on both test-content (out-of-distribution, OOD) and test-repetition: the largest model achieved 99.6% success with 23.8 mm MPJPE-L on test-content, compared to 98.0% success with 27.7 mm for the smallest (1.2M). Gains were most pronounced on OOD motions, confirming that scale improves generalization. For compute, more GPUs yielded better asymptotic performance at the same iteration count, as larger batch sizes improved optimization stability. Shaded regions denote ± 1 standard deviation across 6 evaluation checkpoints. Visualizations of out-of-distribution test motions, including successful and failed tracking cases, are provided in the Supplementary Materials (Fig. S3). We also demonstrated robustness to strong external perturbations in Fig. S5.

Comparison with Other Motion Trackers. We compared against state-of-the-art trackers: GMT [33], Any2Track [30], and BeyondMimic [29]. These baselines were trained on different source datasets (Any2Track and BeyondMimic on LaFAN; GMT on AMASS); for single-motion trackers like BeyondMimic, we retrained for multi-motion tracking using the publicly released code. To maximize protocol consistency, all methods were evaluated in MuJoCo [46] under the same termination criterion defined before (Fig. 2(D–G)). This comparison primarily reflects cross-dataset generalization and scaling effects rather than a fully data-matched benchmark, as the baselines were trained on different source data and retargeting pipelines than `SONIC`. Under this setting, `SONIC` achieved 98.5%/99.2%/97.2% success on test-content/test-repetition/PHUMA [43], compared to 82.0%/85.4%/73.8% for BeyondMimic and 61.6%/69.4%/78.5% for Any2Track. On tracking accuracy, `SONIC` achieved 23.7 mm MPJPE-L, a 42% reduction over BeyondMimic (40.9 mm). The 97.2% success rate on PHUMA is particularly notable because PHUMA aggregates motions from video-based pose estimation with a different retargeting pipeline [43], making it substantially more out-of-distribution than our own held-out splits.

Comparison with Specialist Baseline. To show that a universal tracker can match or exceed specialist controllers, we compared `SONIC` against OpenHomie [13], a state-of-the-art single-task locomotion controller optimized for upper body inverse kinematics control and lower-body velocity tracking. We evaluated both systems on sim-to-sim velocity tracking in MuJoCo across the 0–5 m/s command range (Fig. 2(H–J)). `SONIC` achieved a 98.5% overall survival rate compared to OpenHomie's 43.0%. OpenHomie's survival rate collapsed beyond $\sim$2 m/s, dropping below 20% beyond $\sim$2.5 m/s, whereas `SONIC` maintained near-100% stability up to $\sim$4 m/s. Notably, OpenHomie was specifically designed and trained for locomotion; `SONIC`'s universal policy, trained on diverse whole-body motion data combined with a motion generator, outperformed it.

Real-World Evaluation. We assessed the real-world performance of `SONIC` by deploying it on 124 diverse motion sequences (Fig. S4). As presented in Fig. 2(K–L), our policy achieved motion imitation in the real world that closely matched its simulation results. The real-world policy achieved 99.2% success rate compared to 100% in simulation, with an overall MPJPE-L of 25.7 mm (vs. 22.3 mm in sim). The sim-to-real gap was smallest for the upper body (22.2 mm real vs. 21.8 mm sim) and largest for the feet (53.7 mm vs. 29.0 mm),

reflecting the difficulty of precise foot placement under real-world contact dynamics.

2.2. Interactive Motion Control

In this section, we demonstrated the scalability and robustness of SONIC in whole-body, real-time interactive control tasks (Movie S2). We presented a kinematic runtime generative motion planner that guided the robot's policy through user interaction. Our approach employed an autoregressive framework that continually regenerated future kinematic motions conditioned on the previous states and incoming user commands. For each planning step, the model generated motion segments lasting between 0.8 s and 2.4 s, where the duration was automatically determined by the neural planner to maximize flexibility and robustness. The planner achieved inference times under 5 ms on a standard laptop and ∼12 ms on a Jetson Orin GPU. Replanning was triggered as frequently as every 100 ms, or immediately when user commands were updated, ensuring highly responsive control.

SONIC supported a variety of applications, including navigation control with arbitrary velocity, direction, and style commands; interactive entertainment tasks such as boxing; and locomotion skills such as squatting, crawling, and kneeling, which are useful for downstream applications like teleoperation, because both our kinematic planner and tracking policy were trained on the same large-scale dataset. Utilizing the scalable nature of SONIC, we noted that all the applications above were specified after training, without retraining the planner or the tracking policy.

For navigation control, SONIC accepted velocity commands ranging from 0.0 m/s to 6.0 m/s, as well as arbitrary direction commands spanning 0 to 360 degrees. We note that 6.0 m/s represents the upper bound of the *command* range; the actual achievable velocity is limited by the tracker's capabilities and is filtered by a critically damped spring model (the Generative Kinematic Motion Planner section). The actual commanded-vs-achieved velocity analysis is presented in Fig. 2(H).

SONIC achieved scalable, responsive, and robust navigation control and supported different styles such as drunken walking, injured walking, happy walking, and stealth walking, as shown in Fig. 3(A). The capacity of the model to generate robust in-betweening motions—the task of smoothly transitioning between motion keyframes or segments—showed the flexibility of SONIC, and the potential for more natural human-robot interaction.

SONIC further extended its versatility to interactive entertainment tasks such as boxing, as illustrated in Fig. 3(C). Existing academic solutions [47, 48] and industrial approaches [49] often utilize a limited collection of boxing clips, requiring a switch between multiple expert models or action labels. This approach leads to discontinuous, unnatural transitions and even pauses. SONIC enabled much more fluid, responsive, and natural motion generation, retaining the robot's full freedom of movement throughout the task.

To enable downstream manipulation or navigation in confined environments, skills such as squatting, kneeling, and crawling are essential. As illustrated in Fig. 3, SONIC supported squatting, kneeling, and crawling. For squatting and kneeling, we allowed the pelvis height to be smoothly controlled from 0.3 m to 0.8 m. For navigation in especially tight spaces, SONIC also enabled crawling. The robot could move omnidirectionally using its elbows and knees at velocities from 0.0 m/s to 0.5 m/s.

2.3. Video Teleoperation and Multi-Modal Control

As shown in Fig. 4, SONIC also supported a real-time, multi-modal control framework, enabled by our universal control policy (Movie S3). A unified motion generation system based on the Generalist Model for human motion (GEM) [50]—a single model that performs both human motion estimation and generation—was designed to generate human motions from three input modalities: videos, natural-language commands, and music audio. The multi-modal motion generation system coordinated smooth transitions among modalities.

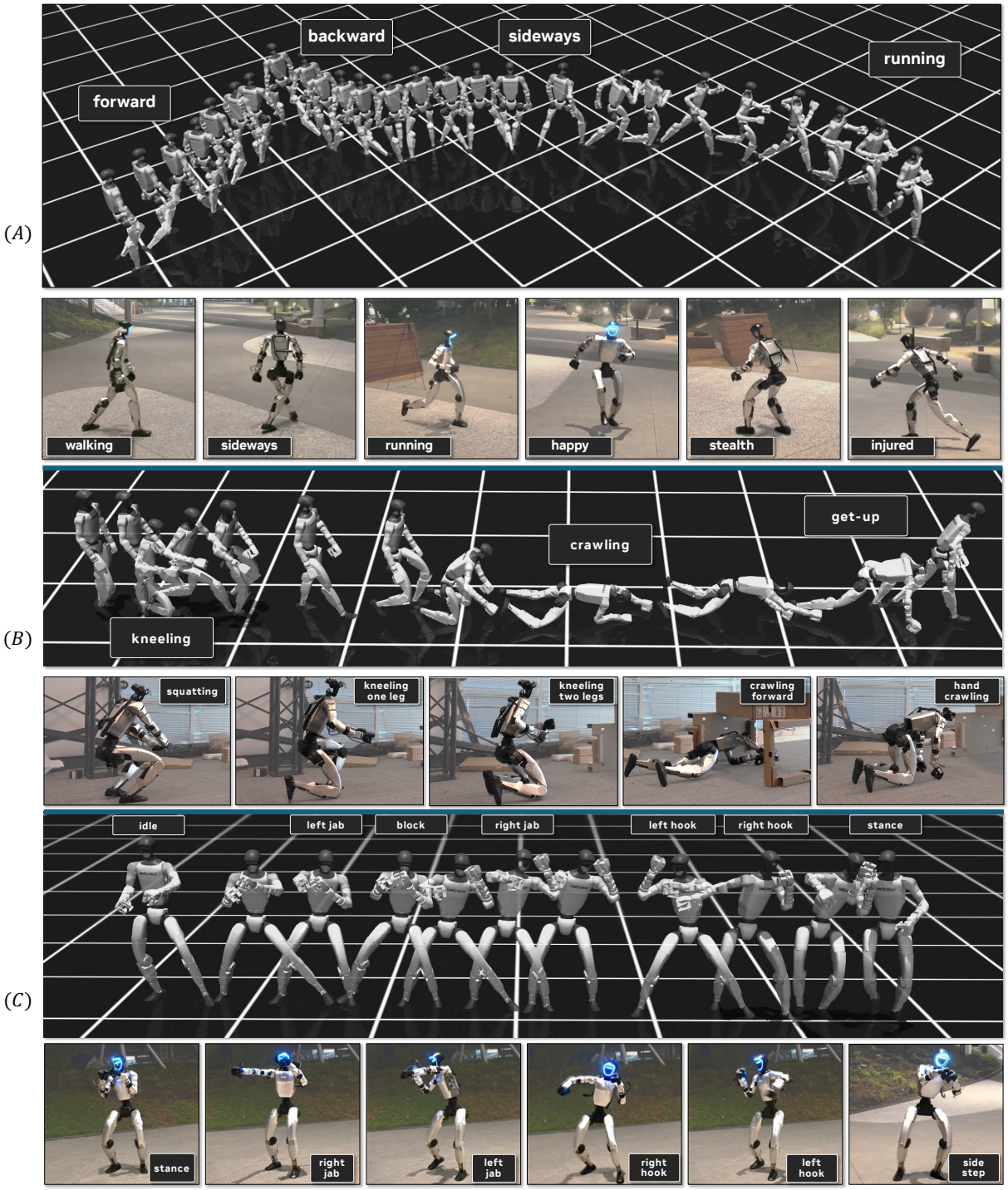

Figure 3: **Interactive whole-body control with** `SONIC`**.** A single universal control policy, driven by the real-time kinematic planner, executes diverse user-commanded behaviors on the Unitree G1 in both simulation and the real world. **(A)** Navigation across velocities, directions, and locomotion styles; **(B)** squatting, kneeling, crawling, and getting up at arbitrary pelvis heights; **(C)** continuous boxing. Each row is a temporally ordered frame sequence.

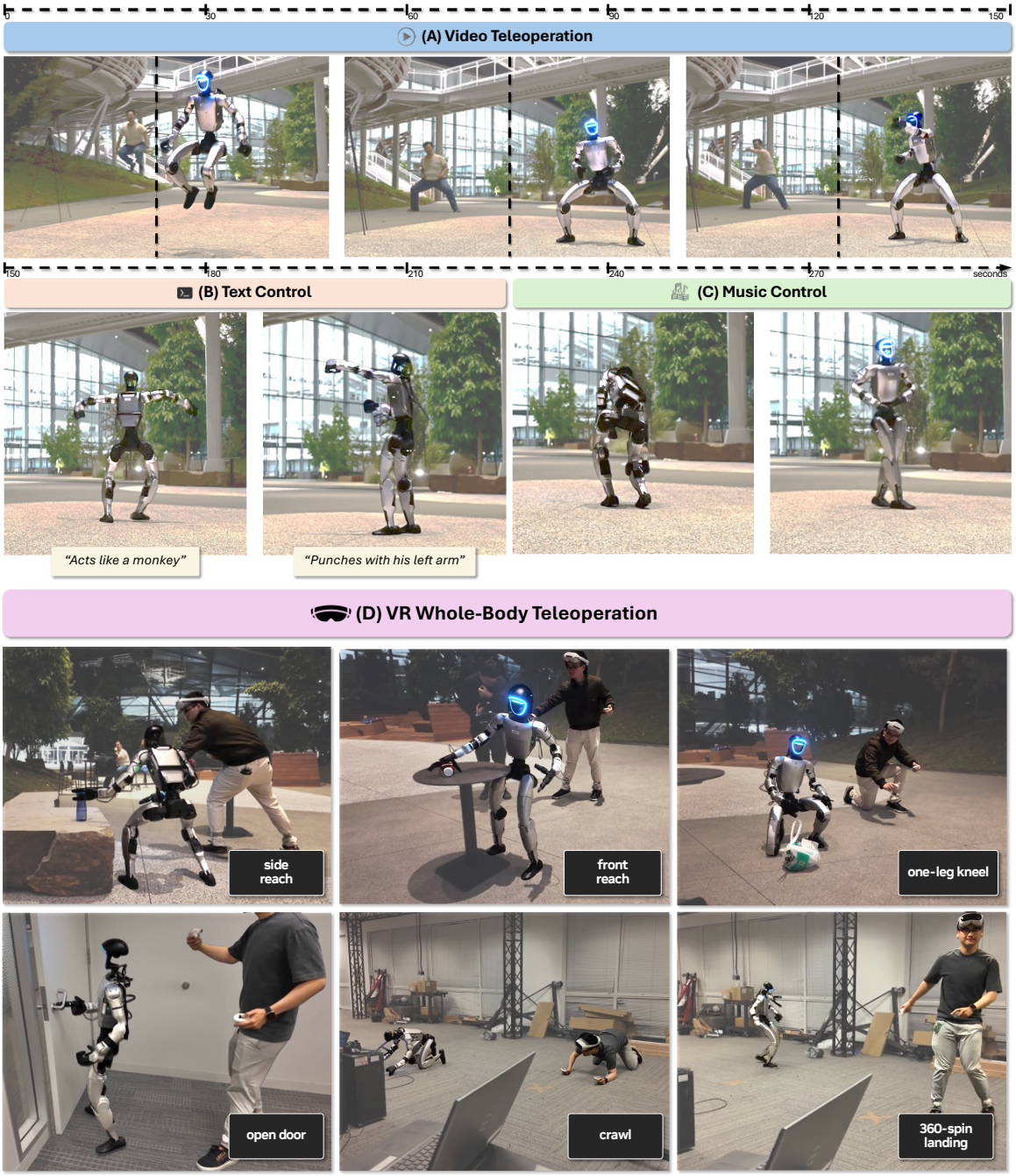

Figure 4: **Multi-modal, real-time control with** SONIC**.** A single universal control policy is driven by heterogeneous input modalities. **(A)** Video teleoperation: full-body motion is estimated from a monocular RGB video stream and tracked in real time. **(B)** Text control: target motions are generated from natural-language prompts. **(C)** Music control: target motions are generated from music, conditioned on melodic and rhythmic structure. **(D)** VR whole-body teleoperation: an operator drives the robot through a range of dynamic whole-body skills. Each panel shows a temporally ordered sequence of frames.

Video Teleoperation. For video control, the system supported both pre-recorded clips and live monocular webcam streams. Human motion was estimated at $\geq$60 frames per second (fps), enabling interactive teleoperation without specialized motion-capture hardware. Video control provided a higher-fidelity specification of pose and timing, yielding a precise imitation of movements.

Music and Text Control. For text control, the system accepted natural-language prompts and synthesized target motions at $\geq$60 fps. An interactive graphical interface supported free-form prompting at any time with immediate on-robot responses (for example, "walk forward", "kick left foot", or "dance like a monkey"). For music control, GEM generated dance motions (which our tracking policy imitated) conditioned on melodic and rhythmic structure, tempo, and musical characteristics. Our system supported transitions between modalities. For example, users could initiate fine-grained control via video, switch to text for general control, and finally hand off to music for performance.

2.4. VR-Based Teleoperation

We built two VR teleoperation interfaces on top of `SONIC` (Movie S4). *Whole-body teleoperation* used a PICO headset, ankle trackers, and handheld controllers [51] to stream full-body poses in the Skinned Multi-Person Linear (SMPL) [52] format, encoded via the human motion encoder $\mathcal{E}_h$. *3-point teleoperation* used only the headset and controllers (no ankle trackers), outputting three upper-body SE(3) poses (head, both wrists), hand joint angles, waist height, and a navigation command; the kinematic planner generated the lower body. Both interfaces used the same universal token space; video-based teleoperation is also supported (the Video Teleoperation and Multi-Modal Control section). We used the VR teleoperation interfaces to collect teleoperation data for training VLA foundation models. Full interface details are provided in the Supplementary Materials (the VR Teleoperation and VLA Integration Details section).

2.5. Foundation-Model-Driven Loco-manipulation

We connected a GR00T N1.5 VLA model [42, 53] to the universal token interface, enabling autonomous whole-body control (Fig. 5, Tab. 1; Movie S5). We evaluated the resulting VLA-driven system on five loco-manipulation tasks of increasing complexity. The first task (apple to plate) used the 3-point interface; the remaining four used the whole-body interface, where the VLA predicted a 78-dimensional action comprising a 64-dimensional universal motion token and 14-dimensional hand joint angles. We observed that predicting universal tokens produced smoother and safer behavior than predicting explicit SMPL poses, which resulted in jerky motions and poor directional control (see the Validation of Key Design Choices section for an ablation). All success rates are strict binary outcomes over 10–20 trials per task (Tab. 1).

Apple to plate (3-point interface): The robot walked to a table, picked up an apple, and placed it on a plate. Trained on 300 trajectories, the policy achieved 90% success over 20 trials.

Object pickup (carrot, scrub; whole-body interface): The robot walked to a table, located a target object, and grasped it under randomized table heights [24–30 inches (60.96–76.2 cm)] and starting positions. Trained on 3,900 trajectories (300 per object, 13 objects), the policy achieved 75% (carrot) and 95% (scrub) success.

Open trash can (whole-body interface): The robot navigated to a trash can and stepped on a pedal to open the lid, requiring precise foot placement and dynamic single-leg balance under closed-loop VLA control. Trained on 200 trajectories, the policy achieved 70% success. This task required the VLA to coordinate full-body dynamics, using feet as manipulators.

Soda can to trash can (whole-body interface): The most complex task, combining five sequential skills: walk to the table, pick up the can with one hand, navigate to the trash can, open the lid by stepping on the pedal with one foot while balancing on the other, and throw the can inside. This required simultaneous hand manipulation, foot manipulation, and dynamic balance within a single action sequence. Trained on 1,000 multi-object trajectories, the policy achieved 60% success.

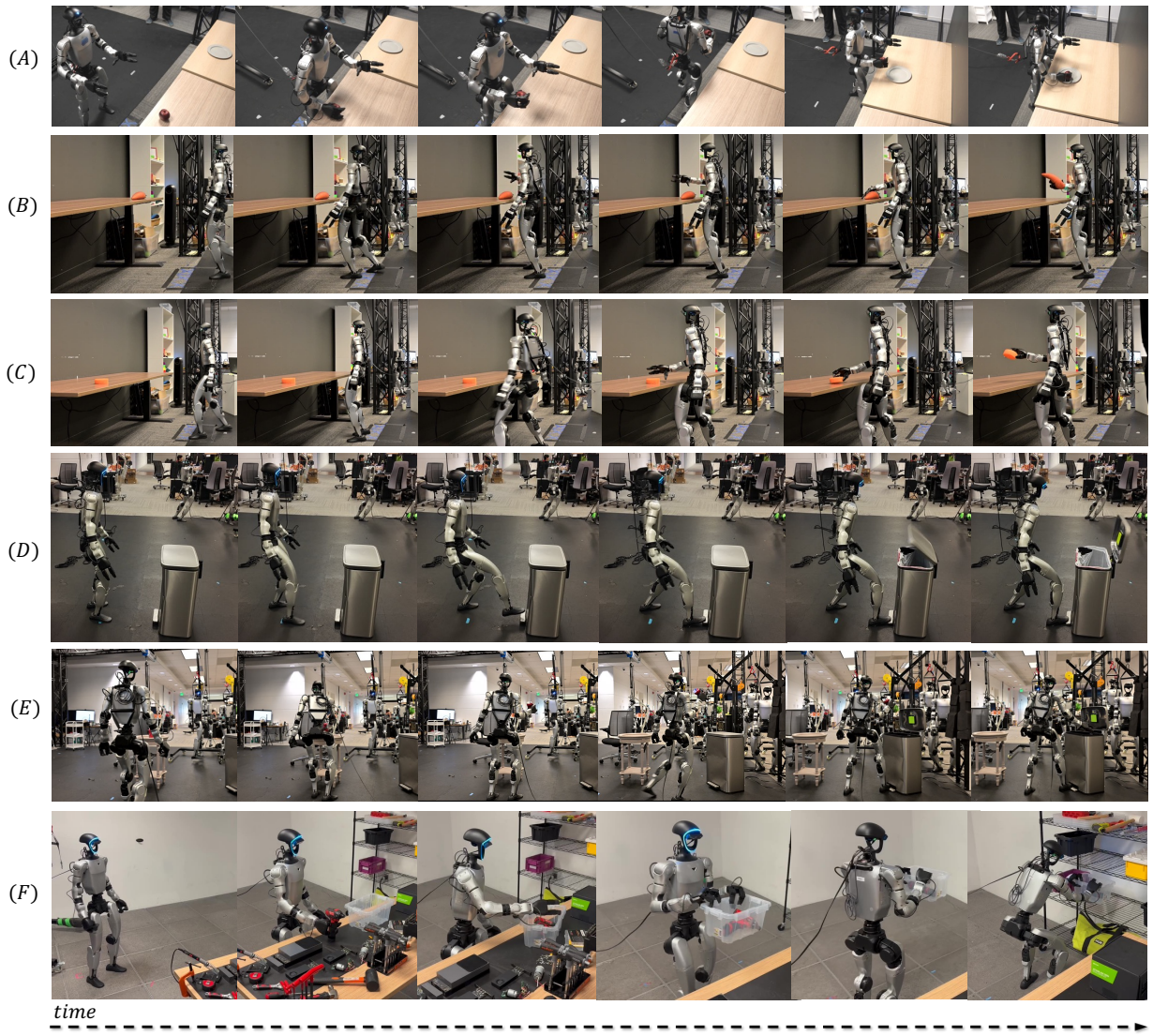

Figure 5: **VLA-driven loco-manipulation tasks.** Each row shows a temporally ordered sequence of frames (time proceeds left to right). **(A)** Apple-to-plate pick-and-place via the 3-point interface. **(B)** Carrot pickup. **(C)** Scrub pickup. **(D)** Open trash can by stepping on the pedal. **(E)** Soda can to trash can: pick up, navigate, step on pedal, and throw. **(F)** Drill and box relocation: pick up drill, place in box, and carry to shelf. See Tab. 1 for success rates.

Drill and box relocation (whole-body interface): A multi-stage task where the robot picked up a drill, placed it in a box, and carried the box to a shelf with both hands. Trained on 300 trajectories, the policy achieved 70% success.

Across all five tasks (10–20 trials each), the VLA achieved 75% average success using the universal token action space. The soda-can and trash-can tasks illustrate autonomous whole-body loco-manipulation with coordinated hand and foot placement, a capability that would be difficult to realize with action spaces that decouple upper-body control from locomotion.

(A) VLA task success rates (universal motion token action space)

Task	Interface	Training Data	Trials	Success
Apple to plate	3-point	300 trajs (single-obj)	20	90%
Object pickup (carrot)	whole-body	3,900 trajs (multi-obj)	20	75%
Object pickup (scrub)	whole-body	3,900 trajs (multi-obj)	20	95%
Open trash can (foot)	whole-body	200 trajs	10	70%
Soda can to trash can	whole-body	1,000 trajs (multi-obj)	10	60%
Drill and box relocation	whole-body	300 trajs	10	70%
Average (5 tasks)				**75%**

(B) Action space: universal motion tokens vs. explicit SMPL poses

Task	FSQ Token	SMPL Poses	Δ
Carrot pickup	**75%**	60%	+15
Open trash can (foot)	**70%**	20%	+50
Soda can to trash can	**60%**	0%	+60
Average	**68%**	27%	**+42**

Table 1: **Vision-language-action (VLA) control through the universal token interface. (A)** Task success rates. A GR00T N1.5 model, fine-tuned on teleoperated data, is evaluated across five whole-body loco-manipulation tasks (the object-pickup variants share one policy and are averaged as a single task for the 5-task mean); success is a strict binary outcome (no partial credit), measured over n=10–20 trials per task. **(B)** Action-space ablation: task completion success rate using universal motion tokens vs. explicit SMPL poses. The compact, structured FSQ token space is substantially easier for the VLA to learn, and the gap widens on more complex tasks (for example, 60% vs. 0% on soda-can-to-trash-can).

2.6. Discussion

We cast motion tracking as a scalable task for learning a single, versatile humanoid controller. By training SONIC on 100 million+ motion frames with up to 128 GPUs, we obtain a single policy that produces natural, robust whole-body behaviors across diverse conditions. Equally important, we build the practical system that makes tracking usable in real deployments: a real-time kinematic motion planner that converts intent into short-horizon reference motions, and a universal token space that unifies heterogeneous interfaces (teleoperation, video, text, and music) within one policy.

We observe consistent improvements as data, model capacity, and compute increase, with generalization to unseen motions in simulation and real-world deployments. These findings support motion tracking as a practical route to acquire broad, transferable whole-body priors without per-task reward engineering.

Why does motion tracking scale. We attribute the favorable scaling properties of motion tracking to its dense, per-frame supervisory signal. Each training frame provides an explicit target pose, so the learning signal remains informative as the dataset grows in size and diversity. This stands in contrast to adversarial imitation methods (AMP [16], ASE [17]), where a discriminator must distinguish real from generated motions across the full distribution; as diversity increases, the discriminator's task becomes harder and its feedback less informative, leading to mode collapse [19, 20]. It also contrasts with task-specific reward engineering (for example, locomotion controllers such as OpenHomie [13]), where each behavior requires a tailored objective that does not generalize. Our comparison with OpenHomie demonstrates this concretely: even on velocity tracking, SONIC's universal tracker achieves 98.5% survival versus OpenHomie's 43.0%, showing that data diversity benefits a universal tracker more than specialization benefits a narrow one. Furthermore, OpenHomie's velocity tracking performance plateaus when scaling beyond 8 GPUs (Fig. S6), whereas SONIC continues to improve with additional compute.

Since the token space represents the full body, VLAs can control the entire kinematic chain, including the feet. Our VLA experiments demonstrate tasks requiring coordinated hand grasping and precise foot placement within a single action sequence.

Limitations include the lack of formal treatment of safety and energy efficiency for extended deployments. The tracker is robust to noisy planner output through domain randomization on motion commands during training and the critically damped spring model that filters unrealistic commands at deployment, but under more extreme conditions or very dynamic motions, the tracker may lose balance.

In summary, scaling motion tracking yields reliable, general whole-body control; pairing it with a planner, a universal token space, and an efficient onboard stack makes it usable as a system. We expect SONIC to serve as a practical foundation upon which higher-level perception and reasoning can be built to advance general-purpose humanoid autonomy.

3. Materials and Methods

Study Design. We evaluated whether physics-based motion tracking scales favorably with data, model size, and compute for humanoid whole-body control. We trained policies in simulation (Isaac Lab [45]) and tested them in both simulation and the real world. We systematically varied data size, model size, and compute (the Motion Tracking section) and evaluated on three held-out test sets with predefined splits (Tab. 2): two from our dataset (test-content, test-repetition) and one external benchmark (PHUMA). Scaling curves report the mean ± 1 standard deviation over $n{=}6$ independent evaluations per configuration. Real-world evaluation covered 124 motion sequences (one trial per sequence; 123 completed successfully). Statistical procedures are detailed in the Statistical Analysis section. VLA task success rates (Tab. 1) were measured over 10–20 trials per task. All training runs used the same hyperparameters (Tab. S2) and reward function (Tab. S3).

3.1. Humanoid Motion Dataset

Our motion dataset was built from a large-scale motion-capture collection. To promote diversity among participants, the collection includes a balanced mix of male and female performers. The dataset spans a broad spectrum of everyday human behaviors, including locomotion, daily activities, gesturing, and a diverse set of combat motions with varied stylistic expressions. Clip durations range from 1 to 180 seconds. In total, the collection covers thousands of unique motion behaviors, with most actions performed by multiple participants across multiple takes, providing rich intra- and inter-participant variation, as can be seen in Fig. S2. The source dataset contains approximately 700 hours of human motion. After retargeting to the Unitree G1 using General Motion Retargeting (GMR) [54] and PyRoki [55], we filtered out physically implausible motions (such as stair climbing and seated activities) that cannot be executed on the target robot, yielding 611 hours of training data (100+ million frames at 50 Hz).

Dataset Diversity and Splits. The dataset spans 33 motion categories (Tab. 2), including basic and advanced locomotion, dance (hip-hop, Latin, vogue, fila), gestures, combat (sword, martial arts, magic), object manipulation (one-handed and two-handed at varying heights and object sizes), tool use (valves, levers, chainsaws, brooms), injured-gait, stylistic variations (drunk, zombie, stealth), role-play, and more. Each motion is captured from multiple participants and is mirrored, yielding paired left/right variants. We constructed explicit train/test splits to enable rigorous evaluation (Tab. 2). The **training set** covers 8,447 unique motion sub-categories (611 hours). The **test-content** split isolates *unseen motion content* (sub-categories entirely absent from training), whereas **test-repetition** isolates *unseen repetitions* of known content (different takes and actor performances).

Public Data Release. A substantial portion of our motion-capture dataset has been publicly released as the BONES-SEED dataset [56], available on Hugging Face. BONES-SEED contains 142,220 annotated motion sequences (288 hours) from 522 actors in SOMA [57] and Unitree G1 formats, with natural language descriptions, temporal segmentation labels, and actor information.

	Train	Test-Content	Test-Rep.
Split statistics			
Clips	317,189	7,016	9,395
Duration (hours)	611	15	12
Unique sub-categories	8,447	182	1,088
Sub-cat. overlap w/ train	—	0%	100%
Clip overlap w/ train	—	0%	0%
Main categories (33 total)			
Locomotion (basic + adv.)	53,255	2,481	2,683
Gestures	37,939	1,488	1,125
Acting / Roleplay	68,742	—	20
Combat (sword, martial arts, and more)	50,162	—	—
Props / Object manip.	14,513	701	253
Dance	9,689	504	485
Injured	9,386	1,167	528
Action / Tool use	9,920	228	322
Others (10+ main cat.)	63,583	429	890

Table 2: Dataset split statistics and main/sub-category distribution. Each *main category* (such as Locomotion and Dance) contains many *sub-categories* describing specific motion types (such as "hip-hop slide" and "injured-leg jog"). **Test-content** evaluates generalization to *unseen sub-categories*: its sub-categories are entirely absent from training. **Test-repetition** evaluates robustness to *unseen performances*: all sub-categories overlap with training, but the specific clips are disjoint.

3.2. Universal Humanoid Motion Tracking

Fig. 6 provides an overview of our approach, SONIC, a universal humanoid motion tracking framework that employs a unified control policy to track diverse motion commands from multiple input formats. A key innovation is its ability to seamlessly handle robot motion, human motion, and hybrid motion (combining

upper-body keypoints with lower-body robot motions) through a shared latent representation. We used various motion generators (kinematic motion planner, VR motion generator, human motion generator (GEM)) to generate motion commands, which enable diverse applications including interactive gamepad control, VR 3-point teleoperation, whole-body teleoperation, video-based teleoperation, and multi-modal control from text and music.

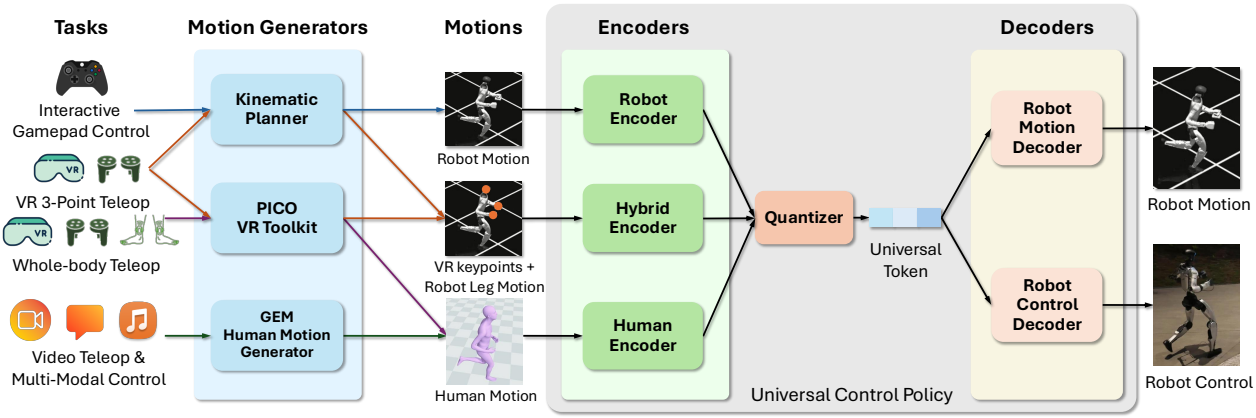

Figure 6: `SONIC` **enables universal humanoid motion tracking through a universal control policy that handles diverse motion commands and modalities.** Specialized encoders process robot, human, and hybrid motion commands into a universal token that drives robot control and motion decoders. This multi-encoder design supports diverse applications including gamepad control, VR teleoperation, whole-body teleoperation, and video teleoperation.

Motion Tracking Formulation. We formulated humanoid motion tracking as a Markov Decision Process $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \gamma \rangle$, comprising state space, action space, transition function, reward function, and discount factor γ. We trained the policy using proximal policy optimization (PPO) [58] to maximize the expected cumulative discounted return $\mathbb{E}\left[\sum_{t=1}^{T} \gamma^{t-1} r_t\right]$. Our environment design followed the general motion tracking formulation [29, 30, 33, 34, 37], and we adapted the well-tuned environmental settings from [29] as the basis for scaling up humanoid motion tracking.

States. The state representation $\boldsymbol{s}_t$ comprises two components: proprioceptive sensing $\boldsymbol{s}_t^{\text{p}}$ and motion command $\boldsymbol{s}_t^{\text{g}}$. Proprioceptive information $\boldsymbol{s}_t^{\text{p}}$ includes joint pose $\boldsymbol{q}_t$, joint velocity $\dot{\boldsymbol{q}}_t$, root angular velocity $\boldsymbol{\omega}_t$, gravity vector $\boldsymbol{g}_t$ in the root frame, and previous action $\boldsymbol{a}_{t-1}$. We concatenated a 10-step history of all proprioceptive quantities and actions into $\boldsymbol{s}_t^{\text{p}}$, that is, $\boldsymbol{s}_t^{\text{p}} \triangleq (\boldsymbol{q}_t, \dot{\boldsymbol{q}}_t, \boldsymbol{\omega}_t, \boldsymbol{g}_t, \boldsymbol{a}_{t-1})_{t-9:t}$, providing the policy with temporal context for anticipatory behavior. The motion command $\boldsymbol{s}_t^{\text{g}}$ has three types: robot motion $\boldsymbol{g}_r$, human motion $\boldsymbol{g}_h$, or hybrid motion $\boldsymbol{g}_m$ (combining upper-body keypoints with lower-body robot motions), where we drop the subscript t for brevity. All state quantities are expressed in the robot's local frame to ensure rotation invariance. We used the 6D rotation representation [59] throughout.

Actions. The policy π outputs target joint positions $\boldsymbol{a}_t$ as actions, which are tracked by proportional-derivative (PD) controllers at each joint. For PD gain settings, we followed prior art [29, 60] that has proven effective in training high-quality tracking policies.

Rewards. We defined the reward as $r_t = \mathcal{R}(\boldsymbol{s}_t^{\text{p}}, \boldsymbol{s}_t^{\text{g}}) + \mathcal{P}(\boldsymbol{s}_t^{\text{p}}, \boldsymbol{a}_t)$, combining a tracking term that minimizes root and body-link pose and velocity errors (including an end-effector term on the head, wrists, and ankles) with penalty terms that encourage smooth, stable motion. The complete reward design is provided in Table S3.

Domain Randomization. To enhance robustness and generalization, we applied systematic domain randomization during training, randomizing physical parameters, periodically perturbing the robot's root velocities to simulate external pushes, and perturbing the target motion commands $\boldsymbol{s}_t^{\text{g}}$. All randomization ranges are

detailed in Table S4.

Universal Control Policy. A main innovation of our tracking framework is its ability to accommodate multiple motion command types from different embodiments through a unified encoder-decoder architecture. Specialized encoders map heterogeneous human and robot motion inputs into a shared latent representation, which is quantized into a *universal token* that drives a common robot control decoder; an auxiliary robot motion decoder facilitates feature learning and serves as an implicit human-to-robot retargeting module. This lets the policy learn from diverse motion sources, including human motion, despite morphological differences.

Encoders. Three specialized encoders process distinct motion command types: the robot motion encoder $\boldsymbol{\mathcal{E}}_r$ encodes robot joint positions and velocities over F_r future frames with a frame interval Δt_r, the human motion encoder $\boldsymbol{\mathcal{E}}_h$ encodes 3D human joint positions [52] over F_h future frames with a frame interval Δt_h, and the hybrid motion encoder $\boldsymbol{\mathcal{E}}_m$ encodes sparse upper-body keypoints (head and hands) of the current frame (for real-time upper-body tracking), combined with lower-body robot motion over F_m future frames with a frame interval Δt_m. Multi-frame inputs enable anticipatory behavior and improve the robustness of the policy. All encoders are implemented as multi-layer perceptrons (MLPs; architecture details in Table S1) that map commands $\boldsymbol{g}_r, \boldsymbol{g}_h, \boldsymbol{g}_m$ into a shared latent space, enabling aligned representations across input modalities.

Quantizer. The encoded latent representation is quantized into a *universal token* $\boldsymbol{z}$ using Finite Scalar Quantization (FSQ) [61], with two tokens, each a D_z-dimensional vector with L_z quantization levels per dimension. We chose FSQ over the vector-quantized variational autoencoder (VQ-VAE) [62] for training stability under joint PPO optimization (the Implementation Details section), and validated this and other design choices in the Validation of Key Design Choices section.

Decoders. The *universal token* $\boldsymbol{z}$ is decoded through two separate decoders. First, a robot control decoder $\boldsymbol{\mathcal{D}}_c$ transforms the *universal token* into motor commands that control the robot's joints. $\boldsymbol{\mathcal{D}}_c$ takes as input the concatenation of the universal token $\boldsymbol{z}$ and the proprioceptive state $\boldsymbol{s}_t^{\text{p}}$, that is, $\boldsymbol{a}_t = \boldsymbol{\mathcal{D}}_c(\boldsymbol{z}, \boldsymbol{s}_t^{\text{p}})$, where all quantities are expressed in the local frame as defined above. The same input representation is used identically during training in simulation and real-world deployment. Second, a robot motion decoder $\boldsymbol{\mathcal{D}}_r$ reconstructs the robot motion command, providing auxiliary supervision to improve the latent space and enhance feature learning. $\boldsymbol{\mathcal{D}}_r$ takes only the universal token as input, that is, $\hat{\boldsymbol{g}}_r = \boldsymbol{\mathcal{D}}_r(\boldsymbol{z})$. Both decoders are implemented as MLPs (Table S1).

Training. We prepared synchronized motion data across all three command types. Each command type $\boldsymbol{g}_r, \boldsymbol{g}_h, \boldsymbol{g}_m$ is encoded via its respective encoder and quantized to produce universal tokens $\boldsymbol{z}_r, \boldsymbol{z}_h, \boldsymbol{z}_m$. For each token, the control decoder $\boldsymbol{\mathcal{D}}_c$ generates motor commands, whereas the motion decoder $\boldsymbol{\mathcal{D}}_r$ reconstructs the robot motion command. The total loss comprises:

$$\mathcal{L} = \mathcal{L}_{\text{ppo}} + \mathcal{L}_{\text{recon}} + \mathcal{L}_{\text{token}} + \mathcal{L}_{\text{cycle}} \tag{1}$$

$$\mathcal{L}_{\text{recon}} = \|\boldsymbol{\mathcal{D}}_r(\boldsymbol{z}_r) - \boldsymbol{g}_r\|^2 + \|\boldsymbol{\mathcal{D}}_r(\boldsymbol{z}_h) - \boldsymbol{g}_r\|^2 + \|\boldsymbol{\mathcal{D}}_r(\boldsymbol{z}_m) - \boldsymbol{g}_r\|^2 \tag{2}$$

$$\mathcal{L}_{\text{token}} = \|\boldsymbol{z}_r - \boldsymbol{z}_h\|^2 + \|\boldsymbol{z}_r - \boldsymbol{z}_m\|^2 + \|\boldsymbol{z}_m - \boldsymbol{z}_h\|^2 \tag{3}$$

$$\mathcal{L}_{\text{cycle}} = \|\boldsymbol{\mathcal{E}}_r(\boldsymbol{\mathcal{D}}_r(\boldsymbol{z}_h)) - \boldsymbol{z}_r\|^2 \tag{4}$$

where $\mathcal{L}_{\text{ppo}}$ denotes the standard PPO loss. $\mathcal{L}_{\text{recon}}$ represents the reconstruction loss for the robot motion command across different input modalities. Notably, when the input command is human motion $\boldsymbol{g}_h$, the encoder-decoder acts as a retargeting pipeline from human to robot motion, and $\mathcal{L}_{\text{recon}}$ serves as a retargeting loss that enables learning from human motion data. $\mathcal{L}_{\text{token}}$ enforces pairwise alignment between all three encoder outputs, ensuring that the same motion produces similar tokens for robot motion, human SMPL poses, or hybrid teleop commands when the source motion is the same. $\mathcal{L}_{\text{cycle}}$ is a cycle consistency loss between the original robot token $\boldsymbol{z}_r$ and the token produced by re-encoding the reconstructed robot motion from the human

token, that is, $\boldsymbol{\mathcal{E}}_r(\boldsymbol{\mathcal{D}}_r(\boldsymbol{z}_h))$. This loss further reinforces latent space coherence, ensuring that the translation from human to robot motion and back preserves the essential motion characteristics.

All four losses are optimized jointly in a single end-to-end training loop. We used asymmetric actor-critic training [63]: the critic observes privileged simulation state (base linear velocity, full body link positions and orientations, and noise-free observations) during training, whereas the actor operates solely on deployment-available observations (noisy proprioceptive sensing and motion commands). The PPO loss updates the encoders, quantizer, and control decoder $\boldsymbol{\mathcal{D}}_c$ (as well as the critic network); the reconstruction, token alignment, and cycle consistency losses update the encoders and motion decoder $\boldsymbol{\mathcal{D}}_r$. Gradients propagate through the FSQ quantizer via straight-through estimation [61], allowing PPO to shape the encoder representations. In practice, the auxiliary losses regularize the latent space and stabilize rather than destabilize PPO optimization, and we observed no training instabilities from coupling quantization with RL across model scales.

We employed bin-based adaptive motion sampling that partitions the dataset into fixed-duration bins and weights sampling by capped failure rates, balancing targeted practice on challenging motions with uniform coverage. We trained using distributed training powered by [64] and [65] across multiple compute nodes in Isaac Lab [45]. Training hyperparameters are provided in Table S2.

Tasks and Applications. The multi-encoder design enables diverse applications through the same policy: interactive gamepad control via the kinematic planner and $\boldsymbol{\mathcal{E}}_r$; VR whole-body and 3-point teleoperation via $\boldsymbol{\mathcal{E}}_h$ and $\boldsymbol{\mathcal{E}}_m$ respectively; VLA-driven autonomous control by predicting universal tokens (the Foundation-Model-Driven Loco-manipulation section); and multi-modal control (video, text, music) via GEM [50] and $\boldsymbol{\mathcal{E}}_h$ (the Multi-modal Motion Generation section).

3.3. Generative Kinematic Motion Planner

Our generative kinematic motion planner is a large-scale latent generative model, trained on the same natural whole-body motion data as the motion tracking policy. At a high level, the planning process was formulated as an autoregressive motion in-betweening generation task, that is, generating the intermediate motion that smoothly connects the current pose to a target keyframe. The context keyframes capture historical robot states, such as joint positions and root positions, whereas target keyframes are either navigation guidance keyframes generated from user commands such as velocity, direction, and style, or skill-specific targets for actions such as squatting, crawling, and boxing.

Motion Representation. During training, we sampled motion segments of length between 0.8 s and 2.4 s, extracting the keyframes at both endpoints to serve as the context and target keyframes. Our motion representation is mathematically equivalent to the humanoid pose configuration q_t as introduced in the Universal Humanoid Motion Tracking section. Specifically, we represented kinematic motion using the pelvis-relative joint positions and global joint rotations. During training, we randomly rotated the training samples to enable planning in all initial orientations. Incorporating global rotation instead of local, canonicalized rotation is essential for generating motions such as squatting and crawling, where the notion of heading is ill-defined and affects the quality of motion planning. We refer readers to [66] and [67] for similar insights and additional discussion.

Generative Neural Backbone in Latent Space. Planning was conducted in the latent space, where continuous motions are first encoded as a sequence of latent tokens as follows:

$$\{z_t\}_{t=1}^{T/4} = \text{enc}\left(\{p_t, r_t\}_{t=1}^{T}\right), \tag{5}$$

where p_t and r_t denote the pose configuration and root position at frame t, respectively. In practice, the encoder operates with a downsampling rate of 4. The latent token sequence is encoded by models such as Transformers or Conv1D networks to capture temporal consistency.

The in-betweening process in the token space is guided by two constraints: the starting and target keyframes, denoted as $\{p_t, r_t\}_{t=1}^{4}$ and $\{p_t, r_t\}_{t=T-4}^{T}$ respectively. Rather than training the network to predict the entire sequence of tokens from these sparse constraints in a single pass, we adopted a masked token prediction approach [68–71]. In this framework, the neural backbone iteratively predicts and finalizes the subset of tokens for which it has the highest confidence, progressively refining the prediction:

$$h = \mathcal{F}\left(\{p_t, r_t\}_{t=1}^{4}, \{p_t, r_t\}_{t=T-4}^{T}, \{z_t\}_{t=1}^{T/4}\right), \tag{6}$$

$$\text{Prob}(z_t) = \sigma(h). \tag{7}$$

This process is iterative, in which $\mathcal{F}(\cdot)$ denotes the neural backbone, and h represents the logits for each token position. Token probabilities are computed by applying a softmax function $\sigma(\cdot)$ to the logits. At the first iteration, all latent tokens are unknown, and we initialized the latent embedding with a learnable mask embedding, z_{masked}. During training, the proportion of masked tokens is uniformly sampled from the range $[100\%, 0\%]$. During inference, a cosine schedule determines the proportion of tokens to finalize at each iteration, specifically $1.0 - \cos\left(\frac{\pi}{2} \cdot \frac{L}{L_{\max}}\right)$, where L is the current iteration and $L_{\max}$ is the maximum number of iterations. After finalization of all tokens, the predicted tokens are used to reconstruct the kinematic motions and generate the robot control signals.

Root trajectory spring model. We proposed to use an intuitive critically damped spring model to generate the root position and heading of the keyframes from user commands as follows:

$$x(t) = \left(x_T - x_0 + \left(v_0 + \frac{c}{2}(x_T - x_0)\right)t\right)e^{-\frac{c}{2}t}, \tag{8}$$

where x_T denotes the target value, x_0 the initial value, v_0 the initial velocity, and c the damping coefficient. We applied this critically damped spring model to three quantities: the pelvis position along the x-axis, the pelvis position along the y-axis, and the projected heading angle of the pelvis. Damping coefficients of $5\ln(2)$ and $20\ln(2)$ were used for position and heading respectively. The target values may be obtained directly from the controllers. Alternatively, if the controller only specifies a desired velocity, we can compute the expected target positions after 1.0 s using the desired velocity. The target keyframes were then placed at the position and heading with $x(1.0)$, as computed by the spring model in Eq. (8). In practice, the planner is robust to the choice of damping coefficients—owing to its variable-length generation (0.8–2.4 s) and strong in-betweening, the spring model can often be omitted entirely—but including it improves behavioral predictability and safeguards against unrealistic commands, such as abruptly reversing direction from 6.0 m/s to -6.0 m/s.

Keyframe Module and Application Integration. Unlike traditional planners that rely on complex target keyframe generation (such as detailed footstep planning), our system specifies keyframes intuitively and with limited manual effort: navigation keyframes are placed along the target root trajectory using style clips, entertainment skills (such as boxing) use the most expressive segment of a matching clip, and interactive manipulation skills (such as squatting and kneeling) retrieve height-conditioned keyframes online. Combined with autoregressive replanning and robust variable-length in-betweening, this lets the planner generate the full distribution of transitional motions for a given skill from only a single clip, generalizing across styles such as walking, running, and crawling. Full per-mode keyframe specification is provided in the Kinematic Planner Details section.

3.4. Multi-modal Motion Generation

For multi-modal control (video, text, music), we adopted GEM [50], a unified generalist model that handles both motion estimation and generation by treating estimation as constrained generation. GEM accepts mixed, time-varying conditions (text, audio, video) and produces human motion sequences via a diffusion-based prior. We integrated GEM with our system using sliding windows with overlap and inpainting-based transitions for

low-latency generation. The generated human motions were fed into SONIC via the human motion encoder $\mathcal{E}_h$.

3.5. Deployment

Experiments were conducted on a Unitree G1 platform (29 actuated joints). All inference ran *onboard* a Jetson Orin GPU using TensorRT with CUDA Graph acceleration, yielding 1–2 ms per policy forward pass and ∼12 ms for motion generation. The system used a multi-rate architecture with four concurrent loops: policy inference at 50 Hz, command streaming at 500 Hz, operator input at 100 Hz, and kinematic planning at 10 Hz. The encoder-decoder design allowed seamless switching between input interfaces (keyboard, gamepad, VR, network streams) by changing the active encoder, with no retraining required. All real-world experiments deployed the largest model (42M parameters). Full deployment details, including the multi-rate architecture, observation gathering pipeline, safety mechanisms, and usage modes, are provided in the Supplementary Materials (the Deployment Architecture section, Fig. S1). Code is available at `https://github.com/NVlabs/GR00T-WholeBodyControl`.

3.6. Validation of Key Design Choices

In this section, we validated key design choices through ablations on the test-content (out-of-distribution) and test-repetition splits, including quantizer design and encoder comparison (Tab. 3), a VLA action space comparison (Tab. 1(B)), latent space alignment analysis (Fig. S7), and kinematic planner validation.

FSQ Tokens vs. Explicit Poses for VLA. A key motivation for quantization is downstream VLA learning. We compared two action spaces (Tab. 1(B)): the VLA predicts FSQ tokens (78-dim: 64-dim token + 14-dim hands), decoded by the universal control policy, vs. the VLA directly predicts SMPL whole-body poses and hand joints (81-dim total). FSQ tokens outperformed SMPL by +42 percentage points on average (68% vs. 27%), with the gap widening on complex tasks (60% vs. 0% on soda-can-to-trash-can). We attributed this to the compactness of the quantized latent space: FSQ tokens provide a low-dimensional, discrete action space that is easier for the VLA to learn from teleoperated demonstrations, whereas the high-dimensional continuous SMPL pose space amplifies small prediction errors into large tracking failures.

Quantizer Design and Configuration. We chose Finite Scalar Quantization (FSQ) [61] over VQ-VAE [62] because FSQ avoids codebook collapse, a failure mode where large portions of a learned codebook are unused. Under our diverse motion distribution (33 categories, 8,447 sub-categories), this can be a major concern. For a fair comparison, we used a multi-head VQ-VAE with comparable capacity (4 heads, codebook size 512, 2 tokens). As shown in Tab. 3(A), FSQ outperformed VQ-VAE by 8.7 mm MPJPE-L on test-content. We also studied the effect of quantizer capacity (Tab. 3(B)) by varying per-token levels and dimensions (all configurations use two tokens). Due to compute constraints, this sweep was run on 32 GPUs rather than 128. For example, FSQ-16-16 denotes 16 quantization levels and 16 dimensions per token. Increasing capacity consistently improved performance, with token dimension having a larger effect than quantization levels, suggesting that representational capacity matters more than quantization granularity for diverse motion tracking. We used FSQ-32-32 as our default configuration throughout the paper.

Multi-Encoder Performance and Consistency Losses. Our multi-encoder design maps three heterogeneous input types (robot motion, human SMPL poses, and hybrid teleop commands) into a shared token space, aligned by the consistency losses $\mathcal{L}_{\text{token}}$ and $\mathcal{L}_{\text{cycle}}$. As shown in Tab. 3(C), all three encoders maintained >99.2% success, with the human encoder showing only a +0.6 mm MPJPE-L gap from the robot encoder despite operating on a different input format. The hybrid encoder showed a larger MPJPE-L (26.5 mm, +2.7 mm from the robot encoder) due to partial observability (only sparse upper-body keypoints). Removing the consistency losses caused an 8× increase in cross-encoder divergence (Fig. S7), confirming they are necessary for cross-encoder alignment. This alignment is critical for downstream VLA learning: since the VLA directly predicts tokens, teleoperation data collected via different encoders (human, hybrid, or robot) should occupy the same latent space to provide the VLA with a consistent training distribution.

	Test-Content (OOD)				Test-Repetition			
Configuration	SR (%)	MPJPE-L	Vel. Dist.	Accel. Dist.	SR (%)	MPJPE-L	Vel. Dist.	Accel. Dist.
(A) Quantizer design (128 GPUs)								
FSQ (ours)	**99.3**	**26.6**	**3.14**	**1.17**	**99.6**	**25.5**	**3.22**	**1.23**
VQ-VAE	98.7	35.3	3.76	1.37	99.3	32.2	3.83	1.44
(B) FSQ configurations (32 GPUs, 2 tokens each)								
FSQ-16-16	96.9	35.7	3.68	1.26	97.5	32.7	3.75	1.31
FSQ-16-32	98.3	29.7	3.39	1.21	98.7	28.4	3.48	1.27
FSQ-32-16	98.3	30.3	3.44	1.22	98.4	28.9	3.52	1.28
FSQ-32-32 (ours)	**98.8**	**27.5**	**3.25**	**1.19**	**99.3**	**26.3**	**3.34**	**1.25**
(C) Encoder comparison (128 GPUs)								
Robot ($\mathcal{E}_r$)	**99.6**	**23.8**	**2.89**	**1.12**	99.8	**22.5**	**2.96**	**1.18**
Human ($\mathcal{E}_h$)	99.6	24.4	3.04	1.24	**99.8**	23.1	3.11	1.30
Hybrid ($\mathcal{E}_m$)	99.2	26.5	3.25	1.22	99.7	25.2	3.31	1.28

Table 3: **Ablation studies.** SR denotes success rate. Each entry reports a single evaluation per configuration on the full test split (descriptive; no statistical test applied). **(A)** FSQ outperforms VQ-VAE by 8.7 mm MPJPE-L on test-content. **(B)** Higher quantizer capacity improves performance; token dimension matters more than levels. **(C)** All encoders maintain >99.2% success; the human encoder shows only a +0.6 mm MPJPE-L gap from the robot encoder.

Role of the Kinematic Motion Planner. The kinematic planner (the Generative Kinematic Motion Planner section) is an application layer that converts high-level user intent into short-horizon kinematic references. Although the tracker is source-agnostic and compatible with alternative planners, ours unifies 25+ distinct skills and styles with a single real-time generative model, each requiring only one representative motion clip and no retraining. The tracker was independently validated on pre-recorded reference motions (the Motion Tracking section), and its robustness extends to planner-generated references through domain randomization on motion commands during training and the spring model that filters unrealistic commands at deployment.

3.7. Statistical Analysis

All statistical tests were two-sided, and P values below 0.001 are reported as $P < 0.001$; exact values, test statistics, and the tabulated data underlying every panel of Fig. 2 are provided in Data S1.

Error bars and uncertainty. Shaded bands in the scaling curves (Fig. 2, A to C) denote the mean ± 1 standard deviation (s.d.) over $n=6$ independent evaluation checkpoints per configuration. Bands in the velocity-tracking comparison (Fig. 2, H to J) denote the mean ± 1 s.d. across repeated locomotion directions per commanded velocity. Success and survival rates are reported together with their underlying trial counts. The tracking metrics (MPJPE-L, velocity distance, and acceleration distance) are means over successfully tracked motions computed on 14 body links; because these metrics are undefined for early-terminated rollouts, no distributional test is applied to them.

Hypothesis tests. Unpaired comparisons used Welch's two-sided t-test (unequal variances): scaling effects were assessed between the smallest and largest configuration on each axis, and success and survival rates were compared on per-motion (or per-run) binary outcomes, which at these sample sizes is equivalent to a two-proportion z-test. Matched comparisons used paired t-tests: velocity-tracking error against the specialist controller was paired across commanded velocities—reported separately for the low-velocity ($\leq$2 m/s) and high-velocity (>2 m/s) regimes, because the two methods cross over—and sim-versus-real success was paired across the 124 real-world evaluation motions. Real-world evaluation used one trial per motion sequence ($N=124$); VLA task success (Tab. 1) was measured over $n=10$–20 trials per task with strict binary scoring. Ablation tables report a single evaluation per configuration and are therefore descriptive.

4. Acknowledgments

We thank Scott Reed, Wei Liu, You Liang Tan, Avnish Narayan, Fengyuan Hu, Yuqi Xie, Letian (Max) Fu, Mengda Xu, Ruijie Zheng, Davis Rempe, Xue Bin (Jason) Peng, Haotian Zhang, Yifeng Jiang, Anna Minx, John Malaska, Chen Tessler, Soha Pouya, Kaushil Prakashbhai Kundalia, Huihua Zhao, Xiaowei Jiang, Olivier Dionne, Michael De Ruyter, Michael Buttner, Qi Wang, Yeongho Seol, Mathis Petrovich, Sanja Fidler, Kaifeng Zhao, Spencer Huang, Gavriel State, Yurong "Kelly" Guo for their thoughtful discussions and help. We thank Amanpreet Singh, Leilee Naderi, Peter Pham, Rajeev Varma, Justin Lee and all the data collection team at GEAR for their support with data collection and providing feedback on teleoperation user experience. We thank Jeremy Chimienti and Tri Cao for their work ensuring robot readiness and conducting the necessary repairs. We also thank Tri Cao, Jazmin Sanchez, Demetria Quijada, and Jesse Yang for their help in filming and editing.

Author contributions. ZL, YY, TW, CL, FC contributed equally as co-first authors; they trained the policies, developed planners, designed the deployment pipelines, and integrated with the VLA. SC, ZC, JL, DM, QB, JP, DS, ZW, XD are core contributors to the framework, including system ID, VLA training, teleoperation, and model optimization. RD, CH, LS, EL, EJ, TH, HX, WX focused on retargeting, filtering the large-scale humanoid motion data, and codebase setup. SY, JK, YC, UI, LF, YZ provided leadership and guidance.

Funding. This research is funded by NVIDIA Corporation.

Competing interests. The authors are or were employees of NVIDIA Corporation at the time of this work. NVIDIA Corporation has filed a patent application related to this work.

Additional qualitative results and videos are available at the project website: https://nvlabs.github.io/GEAR-SONIC/.

Data, Code and Materials availability. The deployment and training code is available at https://github.com/NVlabs/GR00T-WholeBodyControl. A substantial portion of the motion-capture dataset has been publicly released as the BONES-SEED dataset, available at https://huggingface.co/datasets/bones-studio/seed (142,220 annotated motion sequences from 522 actors). An archival snapshot of the code, together with the tabulated data underlying all figures (Data S1), is deposited at Zenodo (DOI: 10.5281/zenodo.21273312). No new materials were generated in this study. The paper, Supplementary Materials, archived code, and BONES-SEED release provide the information and data necessary to assess the reported conclusions.

References

[1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023. 1

[2] Aditya Ramesh, Prafulla Dhariwal, Alex Nichol, Casey Chu, and Mark Chen. Hierarchical text-conditional image generation with clip latents. *arXiv preprint arXiv:2204.06125*, 1(2):3, 2022. 1

[3] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10684–10695, 2022.

[4] Tim Brooks, Bill Peebles, Connor Holmes, Will DePue, Yufei Guo, Leo Jing, David Schnurr, Joe Taylor, Troy Luhman, Eric Luhman, et al. Video generation models as world simulators. *OpenAI Blog*, 1(8):1, 2024.

[5] Jonathan Ho, William Chan, Chitwan Saharia, Jay Whang, Ruiqi Gao, Alexey Gritsenko, Diederik P Kingma, Ben Poole, Mohammad Norouzi, David J Fleet, et al. Imagen video: High definition video generation with diffusion models. *arXiv preprint arXiv:2210.02303*, 2022.

[6] Andreas Blattmann, Tim Dockhorn, Sumith Kulal, Daniel Mendelevitch, Maciej Kilian, Dominik Lorenz, Yam Levi, Zion English, Vikram Voleti, Adam Letts, et al. Stable video diffusion: Scaling latent video diffusion models to large datasets. *arXiv preprint arXiv:2311.15127*, 2023. 1

[7] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020. 1

[8] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Thomas Hennigan, Eric Noland, Katherine Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karén Simonyan, Erich Elsen, Oriol Vinyals, Jack Rae, and Laurent Sifre. An empirical analysis of compute-optimal large language model training. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems*, volume 35, pages 30016–30030. Curran Associates, Inc., 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/c1e2faff6f588870935f114ebe04a3e5-Paper-Conference.pdf.

[9] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, Ed H. Chi, Tatsunori Hashimoto, Oriol Vinyals, Percy Liang, Jeff Dean, and William Fedus. Emergent abilities of large language models. *Transactions on Machine Learning Research*, 2022. ISSN 2835-8856. URL https://openreview.net/forum?id=yzkSU5zdwD. 1

[10] Tairan He, Jiawei Gao, Wenli Xiao, Yuanhang Zhang, Zi Wang, Jiashun Wang, Zhengyi Luo, Guanqi He, Nikhil Sobanbabu, Chaoyi Pan, Zeji Yi, Guannan Qu, Kris Kitani, Jessica K. Hodgins, Linxi Fan, Yuke Zhu, Changliu Liu, and Guanya Shi. ASAP: Aligning Simulation and Real-World Physics for Learning Agile Humanoid Whole-Body Skills. In *Proceedings of Robotics: Science and Systems*, Los Angeles, CA, USA, June 2025. doi: 10.15607/RSS.2025.XXI.066. 1

[11] Xialin He, Runpei Dong, Zixuan Chen, and Saurabh Gupta. Learning Getting-Up Policies for Real-World Humanoid Robots. In *Proceedings of Robotics: Science and Systems*, Los Angeles, CA, USA, June 2025. doi: 10.15607/RSS.2025.XXI.063. 1

[12] Tao Huang, Junli Ren, Huayi Wang, Zirui Wang, Qingwei Ben, Muning Wen, Xiao Chen, Jianan Li, and Jiangmiao Pang. Learning Humanoid Standing-up Control across Diverse Postures. In *Proceedings of Robotics: Science and Systems*, Los Angeles, CA, USA, June 2025. doi: 10.15607/RSS.2025.XXI.064. 1

[13] Qingwei Ben, Feiyu Jia, Jia Zeng, Junting Dong, Dahua Lin, and Jiangmiao Pang. HOMIE: Humanoid Loco-Manipulation with Isomorphic Exoskeleton Cockpit. In *Proceedings of Robotics: Science and Systems*, Los Angeles, CA, USA, June 2025. doi: 10.15607/RSS.2025.XXI.070. 1, 4, 5, 12, 35

[14] Jialong Li, Xuxin Cheng, Tianshu Huang, Shiqi Yang, Ri-Zhao Qiu, and Xiaolong Wang. AMO: Adaptive Motion Optimization for Hyper-Dexterous Humanoid Whole-Body Control. In *Proceedings of Robotics: Science and Systems*, Los Angeles, CA, USA, June 2025. doi: 10.15607/RSS.2025.XXI.061.

[15] Yanjie Ze, Zixuan Chen, Joao Pedro Araujo, Zi ang Cao, Xue Bin Peng, Jiajun Wu, and Karen Liu. TWIST: Teleoperated whole-body imitation system. In *9th Annual Conference on Robot Learning*, 2025. URL https://openreview.net/forum?id=htgNQHa6Ta. 1

[16] Xue Bin Peng, Ze Ma, Pieter Abbeel, Sergey Levine, and Angjoo Kanazawa. Amp: Adversarial motion priors for stylized physics-based character control. *ACM Transactions on Graphics (ToG)*, 40(4):1–20, 2021. 1, 12

[17] Xue Bin Peng, Yunrong Guo, Lina Halper, Sergey Levine, and Sanja Fidler. Ase: Large-scale reusable adversarial skill embeddings for physically simulated characters. *ACM Transactions On Graphics (TOG)*, 41(4):1–17, 2022. 2, 12

[18] Chen Tessler, Yoni Kasten, Yunrong Guo, Shie Mannor, Gal Chechik, and Xue Bin Peng. Calm: Conditional adversarial latent models for directable virtual characters. In *ACM SIGGRAPH 2023 conference proceedings*, pages 1–9, 2023. 2

[19] Chen Tessler, Yunrong Guo, Ofir Nabati, Gal Chechik, and Xue Bin Peng. Maskedmimic: Unified physics-based character control through masked motion inpainting. *ACM Transactions On Graphics (TOG)*, 43(6): 1–21, 2024. 2, 12

[20] Zhengyi Luo, Jinkun Cao, Josh Merel, Alexander Winkler, Jing Huang, Kris Kitani, and Weipeng Xu. Universal humanoid motion representations for physics-based control. In *International Conference on Learning Representations*, volume 2024, pages 56766–56782, 2024. 2, 12

[21] brian ichter, Anthony Brohan, Yevgen Chebotar, Chelsea Finn, Karol Hausman, Alexander Herzog, Daniel Ho, Julian Ibarz, Alex Irpan, Eric Jang, Ryan Julian, Dmitry Kalashnikov, Sergey Levine, Yao Lu, Carolina Parada, Kanishka Rao, Pierre Sermanet, Alexander T Toshev, Vincent Vanhoucke, Fei Xia, Ted Xiao, Peng Xu, Mengyuan Yan, Noah Brown, Michael Ahn, Omar Cortes, Nicolas Sievers, Clayton Tan, Sichun Xu, Diego Reyes, Jarek Rettinghouse, Jornell Quiambao, Peter Pastor, Linda Luu, Kuang-Huei Lee, Yuheng Kuang, Sally Jesmonth, Nikhil J. Joshi, Kyle Jeffrey, Rosario Jauregui Ruano, Jasmine Hsu, Keerthana Gopalakrishnan, Byron David, Andy Zeng, and Chuyuan Kelly Fu. Do as i can, not as i say: Grounding language in robotic affordances. In Karen Liu, Dana Kulic, and Jeff Ichnowski, editors, *Proceedings of The 6th Conference on Robot Learning*, volume 205 of *Proceedings of Machine Learning Research*, pages 287–318. PMLR, 14–18 Dec 2023. URL https://proceedings.mlr.press/v205/ichter23a.html. 2

[22] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alexander Herzog, Jasmine Hsu, Julian Ibarz, Brian Ichter, Alex Irpan, Tomas Jackson, Sally Jesmonth, Nikhil Joshi, Ryan Julian, Dmitry Kalashnikov, Yuheng Kuang, Isabel Leal, Kuang-Huei Lee, Sergey Levine, Yao Lu, Utsav Malla, Deeksha Manjunath, Igor Mordatch, Ofir Nachum, Carolina Parada, Jodilyn Peralta, Emily Perez, Karl Pertsch, Jornell Quiambao, Kanishka Rao, Michael S Ryoo, Grecia Salazar, Pannag R Sanketi, Kevin Sayed, Jaspiar Singh, Sumedh Sontakke, Austin

Stone, Clayton Tan, Huong Tran, Vincent Vanhoucke, Steve Vega, Quan H Vuong, Fei Xia, Ted Xiao, Peng Xu, Sichun Xu, Tianhe Yu, and Brianna Zitkovich. RT-1: Robotics Transformer for Real-World Control at Scale. In *Proceedings of Robotics: Science and Systems*, Daegu, Republic of Korea, July 2023. doi: 10.15607/RSS.2023.XIX.025.

[23] Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, Quan Vuong, Vincent Vanhoucke, Huong Tran, Radu Soricut, Anikait Singh, Jaspiar Singh, Pierre Sermanet, Pannag R. Sanketi, Grecia Salazar, Michael S. Ryoo, Krista Reymann, Kanishka Rao, Karl Pertsch, Igor Mordatch, Henryk Michalewski, Yao Lu, Sergey Levine, Lisa Lee, Tsang-Wei Edward Lee, Isabel Leal, Yuheng Kuang, Dmitry Kalashnikov, Ryan Julian, Nikhil J. Joshi, Alex Irpan, Brian Ichter, Jasmine Hsu, Alexander Herzog, Karol Hausman, Keerthana Gopalakrishnan, Chuyuan Fu, Pete Florence, Chelsea Finn, Kumar Avinava Dubey, Danny Driess, Tianli Ding, Krzysztof Marcin Choromanski, Xi Chen, Yevgen Chebotar, Justice Carbajal, Noah Brown, Anthony Brohan, Montserrat Gonzalez Arenas, and Kehang Han. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In Jie Tan, Marc Toussaint, and Kourosh Darvish, editors, *Proceedings of The 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 2165–2183. PMLR, 06–09 Nov 2023. URL https://proceedings.mlr.press/v229/zitkovich23a.html.

[24] Abby O'Neill, Abdul Rehman, Abhiram Maddukuri, Abhishek Gupta, Abhishek Padalkar, Abraham Lee, Acorn Pooley, Agrim Gupta, Ajay Mandlekar, Ajinkya Jain, Albert Tung, Alex Bewley, Alex Herzog, Alex Irpan, Alexander Khazatsky, Anant Rai, Anchit Gupta, Andrew Wang, Anikait Singh, Animesh Garg, Aniruddha Kembhavi, Annie Xie, Anthony Brohan, Antonin Raffin, Archit Sharma, Arefeh Yavary, Arhan Jain, Ashwin Balakrishna, Ayzaan Wahid, Ben Burgess-Limerick, Beomjoon Kim, Bernhard Schölkopf, Blake Wulfe, Brian Ichter, Cewu Lu, Charles Xu, Charlotte Le, Chelsea Finn, Chen Wang, Chenfeng Xu, Cheng Chi, Chenguang Huang, Christine Chan, Christopher Agia, Chuer Pan, Chuyuan Fu, Coline Devin, Danfei Xu, Daniel Morton, Danny Driess, Daphne Chen, Deepak Pathak, Dhruv Shah, Dieter Büchler, Dinesh Jayaraman, Dmitry Kalashnikov, Dorsa Sadigh, Edward Johns, Ethan Foster, Fangchen Liu, Federico Ceola, Fei Xia, Feiyu Zhao, Freek Stulp, Gaoyue Zhou, Gaurav S. Sukhatme, Gautam Salhotra, Ge Yan, Gilbert Feng, Giulio Schiavi, Glen Berseth, Gregory Kahn, Guanzhi Wang, Hao Su, Hao-Shu Fang, Haochen Shi, Henghui Bao, Heni Ben Amor, Henrik I Christensen, Hiroki Furuta, Homer Walke, Hongjie Fang, Huy Ha, Igor Mordatch, Ilija Radosavovic, Isabel Leal, Jacky Liang, Jad Abou-Chakra, Jaehyung Kim, Jaimyn Drake, Jan Peters, Jan Schneider, Jasmine Hsu, Jeannette Bohg, Jeffrey Bingham, Jeffrey Wu, Jensen Gao, Jiaheng Hu, Jiajun Wu, Jialin Wu, Jiankai Sun, Jianlan Luo, Jiayuan Gu, Jie Tan, Jihoon Oh, Jimmy Wu, Jingpei Lu, Jingyun Yang, Jitendra Malik, João Silvério, Joey Hejna, Jonathan Booher, Jonathan Tompson, Jonathan Yang, Jordi Salvador, Joseph J. Lim, Junhyek Han, Kaiyuan Wang, Kanishka Rao, Karl Pertsch, Karol Hausman, Keegan Go, Keerthana Gopalakrishnan, Ken Goldberg, Kendra Byrne, Kenneth Oslund, Kento Kawaharazuka, Kevin Black, Kevin Lin, Kevin Zhang, Kiana Ehsani, Kiran Lekkala, Kirsty Ellis, Krishan Rana, Krishnan Srinivasan, Kuan Fang, Kunal Pratap Singh, Kuo-Hao Zeng, Kyle Hatch, Kyle Hsu, Laurent Itti, Lawrence Yunliang Chen, Lerrel Pinto, Li Fei-Fei, Liam Tan, Linxi Jim Fan, Lionel Ott, Lisa Lee, Luca Weihs, Magnum Chen, Marion Lepert, Marius Memmel, Masayoshi Tomizuka, Masha Itkina, Mateo Guaman Castro, Max Spero, Maximilian Du, Michael Ahn, Michael C. Yip, Mingtong Zhang, Mingyu Ding, Minho Heo, Mohan Kumar Srirama, Mohit Sharma, Moo Jin Kim, Naoaki Kanazawa, Nicklas Hansen, Nicolas Heess, Nikhil J Joshi, Niko Suenderhauf, Ning Liu, Norman Di Palo, Nur Muhammad Mahi Shafiullah, Oier Mees, Oliver Kroemer, Osbert Bastani, Pannag R Sanketi, Patrick Tree Miller, Patrick Yin, Paul Wohlhart, Peng Xu, Peter David Fagan, Peter Mitrano, Pierre Sermanet, Pieter Abbeel, Priya Sundaresan, Qiuyu Chen, Quan Vuong, Rafael Rafailov, Ran Tian, Ria Doshi, Roberto Martín-Martín, Rohan Baijal, Rosario Scalise, Rose Hendrix, Roy Lin, Runjia Qian, Ruohan Zhang, Russell Mendonca, Rutav Shah, Ryan Hoque, Ryan Julian, Samuel Bustamante, Sean Kirmani, Sergey Levine, Shan Lin, Sherry Moore, Shikhar Bahl, Shivin Dass, Shubham Sonawani, Shuran Song, Sichun Xu, Siddhant Haldar, Siddharth Karamcheti, Simeon Adebola, Simon Guist, Soroush Nasiriany, Stefan Schaal, Stefan Welker,

Stephen Tian, Subramanian Ramamoorthy, Sudeep Dasari, Suneel Belkhale, Sungjae Park, Suraj Nair, Suvir Mirchandani, Takayuki Osa, Tanmay Gupta, Tatsuya Harada, Tatsuya Matsushima, Ted Xiao, Thomas Kollar, Tianhe Yu, Tianli Ding, Todor Davchev, Tony Z. Zhao, Travis Armstrong, Trevor Darrell, Trinity Chung, Vidhi Jain, Vincent Vanhoucke, Wei Zhan, Wenxuan Zhou, Wolfram Burgard, Xi Chen, Xiaolong Wang, Xinghao Zhu, Xinyang Geng, Xiyuan Liu, Xu Liangwei, Xuanlin Li, Yao Lu, Yecheng Jason Ma, Yejin Kim, Yevgen Chebotar, Yifan Zhou, Yifeng Zhu, Yilin Wu, Ying Xu, Yixuan Wang, Yonatan Bisk, Yoonyoung Cho, Youngwoon Lee, Yuchen Cui, Yue Cao, Yueh-Hua Wu, Yujin Tang, Yuke Zhu, Yunchu Zhang, Yunfan Jiang, Yunshuang Li, Yunzhu Li, Yusuke Iwasawa, Yutaka Matsuo, Zehan Ma, Zhuo Xu, Zichen Jeff Cui, Zichen Zhang, and Zipeng Lin. Open x-embodiment: Robotic learning datasets and rt-x models. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6892–6903, 2024. doi: 10.1109/ICRA57147.2024.10611477.

[25] Alexander Khazatsky, Karl Pertsch, Suraj Nair, Ashwin Balakrishna, Sudeep Dasari, Siddharth Karamcheti, Soroush Nasiriany, Mohan Kumar Srirama, Lawrence Yunliang Chen, Kirsty Ellis, Peter David Fagan, Joey Hejna, Masha Itkina, Marion Lepert, Yecheng Jason Ma, Patrick Tree Miller, Jimmy Wu, Suneel Belkhale, Shivin Dass, Huy Ha, Arhan Jain, Abraham Lee, Youngwoon Lee, Marius Memmel, Sungjae Park, Ilija Radosavovic, Kaiyuan Wang, Albert Zhan, Kevin Black, Cheng Chi, Kyle Beltran Hatch, Shan Lin, Jingpei Lu, Jean Mercat, Abdul Rehman, Pannag R Sanketi, Archit Sharma, Cody Simpson, Quan Vuong, Homer Rich Walke, Blake Wulfe, Ted Xiao, Jonathan Heewon Yang, Arefeh Yavary, Tony Z. Zhao, Christopher Agia, Rohan Baijal, Mateo Guaman Castro, Daphne Chen, Qiuyu Chen, Trinity Chung, Jaimyn Drake, Ethan Paul Foster, Jensen Gao, David Antonio Herrera, Minho Heo, Kyle Hsu, Jiaheng Hu, Donovon Jackson, Charlotte Le, Yunshuang Li, Roy Lin, Zehan Ma, Abhiram Maddukuri, Suvir Mirchandani, Daniel Morton, Tony Nguyen, Abigail O'Neill, Rosario Scalise, Derick Seale, Victor Son, Stephen Tian, Emi Tran, Andrew E. Wang, Yilin Wu, Annie Xie, Jingyun Yang, Patrick Yin, Yunchu Zhang, Osbert Bastani, Glen Berseth, Jeannette Bohg, Ken Goldberg, Abhinav Gupta, Abhishek Gupta, Dinesh Jayaraman, Joseph J Lim, Jitendra Malik, Roberto Martín-Martín, Subramanian Ramamoorthy, Dorsa Sadigh, Shuran Song, Jiajun Wu, Michael C. Yip, Yuke Zhu, Thomas Kollar, Sergey Levine, and Chelsea Finn. DROID: A Large-Scale In-The-Wild Robot Manipulation Dataset. In *Proceedings of Robotics: Science and Systems*, Delft, Netherlands, July 2024. doi: 10.15607/RSS.2024.XX.120. 2

[26] Naureen Mahmood, Nima Ghorbani, Nikolaus F Troje, Gerard Pons-Moll, and Michael J Black. Amass: Archive of motion capture as surface shapes. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 5442–5451, 2019. 2

[27] Abhinanda R Punnakkal, Arjun Chandrasekaran, Nikos Athanasiou, Alejandra Quiros-Ramirez, and Michael J Black. Babel: Bodies, action and behavior with english labels. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 722–731, 2021.

[28] Ruilong Li, Shan Yang, David A Ross, and Angjoo Kanazawa. Ai choreographer: Music conditioned 3d dance generation with aist++. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 13401–13412, 2021. 2

[29] Qiayuan Liao, Takara E Truong, Xiaoyu Huang, Yuman Gao, Guy Tevet, Koushil Sreenath, and C Karen Liu. Beyondmimic: From motion tracking to versatile humanoid control via guided diffusion. *arXiv preprint arXiv:2508.08241*, 2025. 2, 3, 5, 14

[30] Zhikai Zhang, Jun Guo, Chao Chen, Jilong Wang, Chenghuai Lin, Yunrui Lian, Han Xue, Zhenrong Wang, Maoqi Liu, Jiangran Lyu, et al. Track any motions under any disturbances. *arXiv preprint arXiv:2509.13833*, 2025. 5, 14

[31] Weishuai Zeng, Shunlin Lu, Kangning Yin, Xiaojie Niu, Minyue Dai, Jingbo Wang, and Jiangmiao Pang. Behavior foundation model for humanoid robots. *arXiv preprint arXiv:2509.13780*, 2025.

[32] Kangning Yin, Weishuai Zeng, Ke Fan, Minyue Dai, Zirui Wang, Qiang Zhang, Zheng Tian, Jingbo Wang, Jiangmiao Pang, and Weinan Zhang. Unitracker: Learning universal whole-body motion tracker for humanoid robots. *IEEE Robotics and Automation Letters*, 11(7):8124–8131, 2026. doi: 10.1109/LRA.2026.3692091.

[33] Zixuan Chen, Mazeyu Ji, Xuxin Cheng, Xuanbin Peng, Xue Bin Peng, and Xiaolong Wang. Gmt: General motion tracking for humanoid whole-body control. *arXiv preprint arXiv:2506.14770*, 2025. 5, 14

[34] Zhengyi Luo, Jinkun Cao, Alexander W. Winkler, Kris Kitani, and Weipeng Xu. Perpetual humanoid control for real-time simulated avatars. In *International Conference on Computer Vision (ICCV)*, 2023. 5, 14

[35] Tingwu Wang, Yunrong Guo, Maria Shugrina, and Sanja Fidler. Unicon: Universal neural controller for physics-based character motion. *arXiv preprint arXiv:2011.15119*, 2020.

[36] Tairan He, Wenli Xiao, Toru Lin, Zhengyi Luo, Zhenjia Xu, Zhenyu Jiang, Jan Kautz, Changliu Liu, Guanya Shi, Xiaolong Wang, et al. Hover: Versatile neural whole-body controller for humanoid robots. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 9989–9996. IEEE, 2025.

[37] Tairan He, Zhengyi Luo, Xialin He, Wenli Xiao, Chong Zhang, Weinan Zhang, Kris M. Kitani, Changliu Liu, and Guanya Shi. Omnih2o: Universal and dexterous human-to-humanoid whole-body teleoperation and learning. In Pulkit Agrawal, Oliver Kroemer, and Wolfram Burgard, editors, *Proceedings of The 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pages 1516–1540. PMLR, 06–09 Nov 2025. URL https://proceedings.mlr.press/v270/he25b.html. 2, 14

[38] Guy Tevet, Sigal Raab, Brian Gordon, Yoni Shafir, Daniel Cohen-Or, and Amit Haim Bermano. Human motion diffusion model. In *The Eleventh International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=SJ1kSyO2jwu. 3

[39] Jianrong Zhang, Yangsong Zhang, Xiaodong Cun, Yong Zhang, Hongwei Zhao, Hongtao Lu, Xi Shen, and Ying Shan. Generating human motion from textual descriptions with discrete representations. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 14730–14740, 2023.

[40] Muhammed Kocabas, Nikos Athanasiou, and Michael J Black. Vibe: Video inference for human body pose and shape estimation. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 5253–5263, 2020.

[41] Li Siyao, Weijiang Yu, Tianpei Gu, Chunze Lin, Quan Wang, Chen Qian, Chen Change Loy, and Ziwei Liu. Bailando: 3d dance generation by actor-critic gpt with choreographic memory. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 11050–11059, 2022. 3

[42] Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi Fan, Yu Fang, Dieter Fox, Fengyuan Hu, Spencer Huang, et al. Gr00t n1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025. 3, 9, 33, 39

[43] Kyungmin Lee, Sibeen Kim, Youngdo Lee, Minho Park, Hyunseung Kim, Dongyoon Hwang, Hojoon Lee, and Jaegul Choo. Phuma: Physically-grounded humanoid locomotion dataset. *arXiv preprint arXiv:2510.26236*, 2025. 3, 4, 5

[44] Unitree. Humanoid robot G1. https://www.unitree.com/g1, 2024. [Accessed 31-10-2025]. 3

[45] Mayank Mittal, Pascal Roth, James Tigue, Antoine Richard, Octi Zhang, Peter Du, Antonio Serrano-Munoz, Xinjie Yao, René Zurbrügg, Nikita Rudin, et al. Isaac lab: A gpu-accelerated simulation framework for multi-modal robot learning. *arXiv preprint arXiv:2511.04831*, 2025. 5, 12, 16

[46] Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033, 2012. doi: 10.1109/IROS.2012.6386109. 5

[47] Sebastian Starke, Yiwei Zhao, Fabio Zinno, and Taku Komura. Neural animation layering for synthesizing martial arts movements. *ACM Trans. Graph.*, 40(4), July 2021. ISSN 0730-0301. doi: 10.1145/3450626.3459881. URL https://doi.org/10.1145/3450626.3459881. 6

[48] Jungdam Won, Deepak Gopinath, and Jessica Hodgins. Control strategies for physically simulated characters performing two-player competitive sports. *ACM Trans. Graph.*, 40(4), July 2021. ISSN 0730-0301. doi: 10.1145/3450626.3459761. URL https://doi.org/10.1145/3450626.3459761. 6

[49] Unitree. Unitree boxing. https://www.unitree.com/boxing, 2025. Accessed: 2025-11-25. 6

[50] Jiefeng Li, Jinkun Cao, Haotian Zhang, Davis Rempe, Jan Kautz, Umar Iqbal, and Ye Yuan. Genmo: A generalist model for human motion. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 11766–11776, October 2025. 6, 16, 17, 39

[51] Zhigen Zhao, Liuchuan Yu, Ke Jing, and Ning Yang. Xrobotoolkit: A cross-platform framework for robot teleoperation. In *2026 IEEE/SICE International Symposium on System Integration (SII)*, pages 15–20. IEEE, 2026. 9, 33, 39

[52] Matthew Loper, Naureen Mahmood, Javier Romero, Gerard Pons-Moll, and Michael J. Black. Smpl: a skinned multi-person linear model. *ACM Trans. Graph.*, 34(6), November 2015. ISSN 0730-0301. doi: 10.1145/2816795.2818013. URL https://doi.org/10.1145/2816795.2818013. 9, 15, 33

[53] Johan Bjorck, Valts Blukis, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi "Jim" Fan, Yu Fang, Dieter Fox, Fengyuan Hu, Spencer Huang, Joel Jang, Xiaowei Jiang, Kaushil Kundalia, Jan Kautz, Zhiqi Li, Kevin Lin, Zongyu Lin, Loic Magne, Yunze Man, Ajay Mandlekar, Avnish Narayan, Soroush Nasiriany, Scott Reed, You Liang Tan, Guanzhi Wang, Jing Wang, Qi Wang, Shihao Wang, Jiannan Xiang, Yuqi Xie, Yinzhen Xu, Seonghyeon Ye, Zhiding Yu, Yizhou Zhao, Zhe Zhang, Ruijie Zheng, and Yuke Zhu. GR00T N1.5: An Improved Open Foundation Model for Generalist Humanoid Robots. https://research.nvidia.com/labs/gear/gr00t-n1_5/, June 2025. Blog post; accessed 2025-10-30. 9, 33, 39

[54] Joao Pedro Araujo, Yanjie Ze, Pei Xu, Jiajun Wu, and C Karen Liu. Retargeting matters: General motion retargeting for humanoid motion tracking. *arXiv preprint arXiv:2510.02252*, 2025. 13

[55] Chung Min Kim, Brent Yi, Hongsuk Choi, Yi Ma, Ken Goldberg, and Angjoo Kanazawa. Pyroki: A modular toolkit for robot kinematic optimization. In *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 1312–1319. IEEE, 2025. 13

[56] Bones Studio. BONES-SEED: Skeletal everyday embodiment dataset. https://huggingface.co/datasets/bones-studio/seed, 2026. 142,220 annotated motion sequences from 522 actors. 13

[57] Jun Saito, Jiefeng Li, Michael de Ruyter, Miguel Guerrero, Edy Lim, Ehsan Hassani, Roger Blanco Ribera, Hyejin Moon, Magdalena Dadela, Marco Di Lucca, et al. Soma: Unifying parametric human body models. *arXiv preprint arXiv:2603.16858*, 2026. 13

[58] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. 14

[59] Yi Zhou, Connelly Barnes, Jingwan Lu, Jimei Yang, and Hao Li. On the continuity of rotation representations in neural networks. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 5745–5753, 2019. 14

[60] Marc Raibert and Farbod Farshidian. Towards dynamic and intelligent humanoids. In *Workshop on Whole-body Control and Bimanual Manipulation: Applications in Humanoids and Beyond, Robotics: Science and Systems (RSS)*, June 2025. 14

[61] Fabian Mentzer, David Minnen, Eirikur Agustsson, and Michael Tschannen. Finite scalar quantization: Vq-vae made simple. In *International Conference on Learning Representations*, volume 2024, pages 51772–51783, 2024. 15, 16, 18, 28

[62] Aaron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. Neural discrete representation learning. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*, NIPS'17, page 6309–6318, Red Hook, NY, USA, 2017. Curran Associates Inc. ISBN 9781510860964. 15, 18, 28

[63] Lerrel Pinto, Marcin Andrychowicz, Peter Welinder, Wojciech Zaremba, and Pieter Abbeel. Asymmetric actor critic for image-based robot learning. In *Proceedings of Robotics: Science and Systems*, Pittsburgh, Pennsylvania, June 2018. doi: 10.15607/RSS.2018.XIV.008. 16

[64] Sylvain Gugger, Lysandre Debut, Thomas Wolf, Philipp Schmid, Zachary Mueller, Sourab Mangrulkar, Marc Sun, and Benjamin Bossan. Accelerate: Training and inference at scale made simple, efficient and adaptable. https://github.com/huggingface/accelerate, 2022. 16

[65] Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, Shengyi Huang, Kashif Rasul, and Quentin Gallouédec. TRL: Transformer Reinforcement Learning. https://github.com/huggingface/trl, 2020. 16

[66] Zichong Meng, Yiming Xie, Xiaogang Peng, Zeyu Han, and Huaizu Jiang. Rethinking diffusion for text-driven human motion generation: Redundant representations, evaluation, and masked autoregression. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 27859–27871, 2025. 16

[67] Zichong Meng, Zeyu Han, Xiaogang Peng, Yiming Xie, and Huaizu Jiang. Absolute coordinates make motion generation easy. *arXiv preprint arXiv:2505.19377*, 2025. 16

[68] Lijun Yu, Yong Cheng, Kihyuk Sohn, José Lezama, Han Zhang, Huiwen Chang, Alexander G Hauptmann, Ming-Hsuan Yang, Yuan Hao, Irfan Essa, et al. Magvit: Masked generative video transformer. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 10459–10469, 2023. 17

[69] Zhuoyan Luo, Fengyuan Shi, Yixiao Ge, Yujiu Yang, Limin Wang, and Ying Shan. Open-magvit2: An open-source project toward democratizing auto-regressive visual generation. *arXiv preprint arXiv:2409.04410*, 2024.

[70] Chuan Guo, Yuxuan Mu, Muhammad Gohar Javed, Sen Wang, and Li Cheng. Momask: Generative masked modeling of 3d human motions. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 1900–1910, 2024.

[71] Ekkasit Pinyoanuntapong, Pu Wang, Minwoo Lee, and Chen Chen. Mmm: Generative masked motion model. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 1546–1555, 2024. 17

Supplementary Materials for Supersizing Motion Tracking for Natural Humanoid Whole-Body Control

Zhengyi Luo[†], Ye Yuan[†], Tingwu Wang[†], Chenran Li[†], Fernando Castañeda[†], Sirui Chen[*], Zi-Ang Cao[*], Jiefeng Li[*], David Minor[*], Qingwei Ben[*], Jinhyung Park[*], David Sami[*], Zi Wang[*], Xingye Da[*], Runyu Ding, Cyrus Hogg, Lina Song, Edy Lim, Eugene Jeong, Tairan He, Haoru Xue, Wenli Xiao, Simon Yuen, Jan Kautz, Yan Chang, Umar Iqbal, Linxi "Jim" Fan[‡], Yuke Zhu[‡]

NVIDIA, 2788 San Tomas Expressway, Santa Clara, CA, USA

[†]Co-first Authors, [*]Core Contributors, [‡]Equal Advising

Contents

S1 Supplementary Methods 28
S2 Supplementary Discussion 33
S3 Supplementary Videos 37
S4 Supplementary Data 39

S1. Supplementary Methods

S1.1. Implementation Details

This section provides details for the network architecture, training hyperparameters, reward definitions, and domain randomization settings.

Network Architecture. Table S1 reports the full configuration of the encoder-decoder architecture used by `SONIC`, including the robot-motion, human-motion, and hybrid encoders; the universal token quantizer parameters (FSQ levels and token dimension); the robot-control and auxiliary robot-motion decoders; and layer dimensions, MLP depth, activation functions, and latent sizes.

Training Hyperparameters. Table S2 lists all training configurations, including PPO hyperparameters, rollout length, and discount factors.

Reward Function. Table S3 provides a complete breakdown of the reward terms used for motion tracking, including root position and orientation matching; body-link position, orientation, and velocity terms; end-effector position tracking; and action rate, joint limit, contact, anti-shake, and feet acceleration penalties.

Domain Randomization. We randomized physical parameters including friction coefficients (μ_s, μ_d), the restitution coefficient (e), first-frame joint positions ($\boldsymbol{q}_0$), and the base center-of-mass position, periodically perturbed the robot's root linear and angular velocities to simulate external pushes, and perturbed the target motion commands $\boldsymbol{s}_t^g$ during training. Table S4 reports the ranges and distributions for all randomization parameters.

Universal Token Quantization. We used Finite Scalar Quantization (FSQ) [61] rather than a vector-quantized variational autoencoder (VQ-VAE) [62] to produce the universal token. FSQ avoids codebook collapse (a failure mode where large portions of the codebook go unused), requires no auxiliary commitment loss or codebook exponential-moving-average (EMA) updates, and provides clean straight-through gradient estimation that is compatible with joint PPO optimization. We validated these design choices in the Validation of Key Design Choices section, including FSQ vs. VQ-VAE, quantizer configuration (levels and dimensions), and multi-encoder alignment.

Module	Architecture	Dims
Network configuration		
Quantizer	FSQ	token dimensions $= D_z$; quantization levels $= L_z$
Encoder (g1)	MLP	hidden$= [2048, 1024, 512, 512]$
Encoder (teleop)	MLP	hidden $= [2048, 1024, 512, 512]$
Encoder (smpl)	MLP	hidden $= [2048, 1024, 512, 512]$
Decoder (actions)	MLP	hidden $= [4096, 4096, 2048, 2048, 1024, 1024, 512, 512]$
Decoder (refs)	MLP	hidden $= [2048, 1024, 512, 512]$
Action dimension	Diagonal Gaussian	29
Critic	MLP	hidden $= [4096, 4096, 2048, 2048, 1024, 1024, 512, 512]$
Motion command		
Future frames	$F_r = F_h = F_m = 10$ frames	
Frame interval	$\Delta t_r = \Delta t_m = 0.1s$, $\Delta t_h = 0.02s$	

Table S1: Universal control policy architecture and hyperparameters.

Training hyperparameter	Value
Num parallel envs per GPU	4096
Num steps per env	24
Learning epochs	5
Num mini-batches	4
Discount γ	0.99
GAE λ	0.95
Clip parameter	0.2
Entropy coefficient	0.013
Value loss coefficient	1.0
Actor learning rate	2×10^{-5}
Critic learning rate	1×10^{-3}
Max gradient norm	0.1
Desired KL	0.01
Adaptive LR min/max	$[1\times10^{-5}, 2\times10^{-4}]$
Init noise std	0.05
Actor std clamp min/max	$[0.001, 0.5]$
Adaptive sampling bin size	1s
Adaptive sampling failure rate cap	$\beta = 200$
Adaptive sampling blending hyperparameter	$\alpha = 0.1$

Table S2: Training hyperparameters.

S1.2. Deployment Architecture

Experiments were conducted on a Unitree G1 platform (29 actuated joints) using the built-in joint-level PD controller. The learned policy output desired joint angles that were passed directly to the PD interface. All components of the inference and management stack ran *onboard*, leveraging the Jetson Orin GPU to minimize feedback latency and improve timing determinism.

Multi-rate architecture. As shown in Fig. S1, the system was organized into four concurrent loops, each running at a rate matched to its role. The **control loop (50 Hz)** assembled observations from the recent robot state and reference motion, invoked the encoder and policy networks, and synthesized joint-space targets; its 50 Hz rate matched the simulation step used during training. The **command writer (500 Hz)** was a separate high-rate process that streamed motor targets through the Unitree low-level API, continuously publishing the latest commands without blocking the policy loop. The **input interface (100 Hz)** sampled operator input independently, supporting seamless switching between keyboard, gamepad, VR controllers, or networked streams. The **kinematic planner (10 Hz)**, when employed, proposed short-horizon reference trajectories based on the operator's high-level commands. Each loop operated on consistent snapshots of the robot state, and we adopted a "latest-data-wins" convention; inter-thread buffers protected by reader-writer locks allowed each consumer to always read the most recent data without blocking producers, so transient delays did not stall the system.

Reward term	Equation	Weight
Tracking rewards $\mathcal{R}(\boldsymbol{s}_t^p, \boldsymbol{s}_t^g)$		
Root position	$r_{\text{pos}}^{\text{root}}(t) = \exp\big(-\|\boldsymbol{p}_{t,r}^p - \boldsymbol{p}_{t,r}^g\|_2^2/0.3^2\big)$	0.5
Root orientation	$r_{\text{ori}}^{\text{root}}(t) = \exp\big(-\|\boldsymbol{o}_{t,r}^p - \boldsymbol{o}_{t,r}^g\|_2^2/0.4^2\big)$	0.5
Body link pos (rel.)	$r_{\text{pos}}^{\text{body}}(t) = \exp\Big(-\frac{1}{\|\mathcal{B}\|}\sum_{b\in\mathcal{B}}\|\boldsymbol{p}_{t,b}^{p,\text{rel}} - \boldsymbol{p}_{t,b}^{g,\text{rel}}\|_2^2/0.3^2\Big)$	1.0
Body link ori (rel.)	$r_{\text{ori}}^{\text{body}}(t) = \exp\Big(-\frac{1}{\|\mathcal{B}\|}\sum_{b\in\mathcal{B}}\|\boldsymbol{o}_{t,b}^{p,\text{rel}} - \boldsymbol{o}_{t,b}^{g,\text{rel}}\|_2^2/0.4^2\Big)$	1.0
Body link lin. vel	$r_{\text{lin}}^{\text{body}}(t) = \exp\Big(-\frac{1}{\|\mathcal{B}\|}\sum_{b\in\mathcal{B}}\|\boldsymbol{v}_{t,b}^p - \boldsymbol{v}_{t,b}^g\|_2^2/1.0^2\Big)$	1.0
Body link ang. vel	$r_{\text{ang}}^{\text{body}}(t) = \exp\Big(-\frac{1}{\|\mathcal{B}\|}\sum_{b\in\mathcal{B}}\|\boldsymbol{\omega}_{t,b}^p - \boldsymbol{\omega}_{t,b}^g\|_2^2/3.14^2\Big)$	1.0
End-effector position	$r_{\text{pos}}^{\text{ee}}(t) = \exp\big(-\frac{1}{5}\sum_{k\in\mathcal{K}}\|\boldsymbol{p}_{t,k}^p - \boldsymbol{p}_{t,k}^g\|_2^2/0.1^2\big)$	2.0
Penalty terms $\mathcal{P}(\boldsymbol{s}_t^p, \boldsymbol{a}_t)$		
Action rate	$r_{\text{act}}(t) = \|\boldsymbol{a}_t - \boldsymbol{a}_{t-1}\|_2^2$	-0.1
Joint limit	$r_{\text{jlim}}(t) = \sum_j \mathbb{1}\left[\boldsymbol{q}_{t,j} \notin [\boldsymbol{q}_{t,j}^{\min}, \boldsymbol{q}_{t,j}^{\max}]\right]$	-10.0
Undesired contacts	$r_{\text{contact}}(t) = \sum_{c\notin\{\text{ankles, wrists}\}} \mathbb{1}[\|\boldsymbol{F}_c\| > 1.0\text{N}]$	-0.1
Anti-shake (ang. vel)	$r_{\text{shake}}(t) = \sum_{k\in\{\text{wrists, head}\}} \|\boldsymbol{\omega}_{t,k}\|_2^2 \cdot \mathbb{1}[\|\boldsymbol{\omega}_{t,k}\| > 1.5]$	$-5\text{e-}3$
Feet acceleration	$r_{\text{feet}}(t) = \sum_{k\in\{\text{ankles}\}} \|\ddot{\boldsymbol{q}}_{t,k}\|_2^2$	$-2.5\text{e-}6$

Table S3: Reward design. Superscript g: goal/target; p: current state; $\mathcal{B}$: tracked body links; $\mathcal{K}$: VR keypoints (head, both wrists, both ankles); "rel": relative to root frame.

Domain Randomization	Sampling Distribution
Physical parameters	
Static friction coefficients	$\mu_s \sim \mathcal{U}[0.3, 1.6]$
Dynamic friction coefficients	$\mu_d \sim \mathcal{U}[0.3, 1.2]$
Restitution coefficient	$e \sim \mathcal{U}[0, 0.5]$
Default joint positions	$\boldsymbol{q}_0 \sim \boldsymbol{q}_0 + \mathcal{U}[-0.01, 0.01]$
Base COM offset (x, y, z)	$\Delta x \sim \mathcal{U}[-0.075, 0.075],\ \Delta y \sim \mathcal{U}[-0.1, 0.1],\ \Delta z \sim \mathcal{U}[-0.1, 0.1]$
Root velocity perturbations (external pushes)	
Root linear vel (x, y, z)	$v_x \sim \mathcal{U}[-0.5, 0.5],\ v_y \sim \mathcal{U}[-0.5, 0.5],\ v_z \sim \mathcal{U}[-0.2, 0.2]$
Push duration	$\Delta t \sim \mathcal{U}[1, 3]$s
Root angular vel	$\omega_{\text{roll}} \sim \mathcal{U}[-0.52, 0.52],\ \omega_{\text{pitch}} \sim \mathcal{U}[-0.52, 0.52],\ \omega_{\text{yaw}} \sim \mathcal{U}[-0.78, 0.78]$
Target motion perturbations $(\boldsymbol{s}_t^g)$	
Target position jitter	$\Delta\boldsymbol{p}^g \sim \mathcal{U}[-0.05, 0.05]^3$ (x,y: ±0.05, z: ±0.01)
Target orientation jitter	$\Delta\phi_{\text{roll}}, \Delta\phi_{\text{pitch}} \sim \mathcal{U}[-0.1, 0.1],\ \Delta\phi_{\text{yaw}} \sim \mathcal{U}[-0.2, 0.2]$
Target linear vel jitter	$\Delta\boldsymbol{v}^g \sim \mathcal{U}[-0.5, 0.5]^3$ (x,y: ±0.5, z: ±0.2)
Target angular vel jitter	$\Delta\omega_{\text{roll}}, \Delta\omega_{\text{pitch}} \sim \mathcal{U}[-0.52, 0.52],\ \Delta\omega_{\text{yaw}} \sim \mathcal{U}[-0.78, 0.78]$
Target joint jitter	$\Delta\boldsymbol{q}_t^g \sim \mathcal{U}[-0.1, 0.1]$

Table S4: Domain randomization parameters applied during training. $\mathcal{U}[\cdot]$: uniform distribution.

Unified motion interface. A central design choice was that the control loop was *source-agnostic* with respect to the reference motion. All motion sources, including pre-recorded clips, planner-generated trajectories, and externally streamed data, populated the same `MotionSequence` structure, which stored per-frame joint positions, joint velocities, root poses, and optionally SMPL body data in flat arrays with frame-indexed accessors. A single shared pointer and a frame counter defined what the policy saw at each tick. Switching between sources was an atomic pointer swap under a mutex, with the frame counter reset to zero. Because the encoder mode was carried as a field on the motion sequence itself, changing the motion source automatically selected the appropriate encoder mode; for example, switching from a pre-recorded clip (mode 0, joint-based encoding) to an externally streamed SMPL sequence (mode 2, SMPL-based encoding) required no explicit mode management by the operator.

State logging and observation gathering. At each control tick, the system first read IMU orientation, angular velocity, and joint positions/velocities from the hardware, remapped them to the policy's joint ordering, and pushed a timestamped snapshot into a dual-purpose state logger. The logger maintained both a fixed-capacity ring buffer for real-time access and per-signal CSV files for offline analysis. History-based observations (such as the past 10 frames of proprioceptive quantities) were retrieved from this ring buffer via strided lookback, with

zero-padding when the buffer contained fewer entries than requested (for example, at startup).

The choice of observations to gather, in what order, and at what offsets was entirely driven by a YAML configuration file. At startup, each entry in the config was matched against a registry of named observation functions, and the system pre-computed a flat list of `(function, offset, dimension)` triples. The control loop then simply iterated this list, calling each function to fill its slice of the observation vector. This design allowed the same binary to serve different policy architectures by swapping a single config file, without recompilation.

GPU inference. Both the interactive kinematic motion planner and the policy inference executed onboard the Jetson Orin GPU using TensorRT with CUDA Graph acceleration: a warmup pass recorded the complete GPU execution graph, and all subsequent calls replayed it with new data, yielding reliable low-variance execution, 1–2 ms per forward pass for the policy and $\sim$12 ms for motion generation.

Heading alignment. When a new motion began, the system captured the robot's current yaw from the IMU and computed a yaw-only rotation that aligned the reference motion's initial heading to the robot's facing direction. This rotation was applied to all subsequent reference root poses, so the robot could execute any motion starting from its current heading. The operator could inject heading corrections at runtime to steer the robot.

Startup and safety. The system followed a three-phase startup: it linearly interpolated the robot from its current configuration to a nominal standing pose over 3 seconds, held until the operator signaled readiness, then activated the policy loop. A joint velocity watchdog triggered an immediate stop if any joint exceeded a predefined threshold. If the encoder's required input data became unavailable during playback, the system automatically fell back to the next available encoder mode, maintaining continuous operation. Upon any stop event, all joints transitioned to a pure-damping mode (K_p=0, K_d=8 Nm·s/rad) that let the robot gently comply rather than holding position or going limp. All motor commands carried CRC32 checksums validated by the firmware before execution.

Data tracking. Pre-recorded reference motions were loaded from CSV directories at startup. The operator selected a clip and triggered playback; the frame counter advanced at 50 Hz, and the policy tracked the reference. When the clip ended, playback paused and the frame reset. This mode was used for evaluation and for motions that required precise, repeatable reference trajectories.

Motion planning. When the planner was enabled, the planner thread generated full-body trajectories at 30 fps temporal resolution (up to 64 frames, $\sim$2 s of predicted motion), replanning at up to 10 Hz, conditioned on a context window of 4 recent frames and operator commands (locomotion mode, target velocity, facing direction). At initialization, the planner context was set to a canonical pose regardless of the robot's actual orientation; subsequent replans built the context from the previously generated trajectory, so the planner autoregressively continued in the coordinate frame established at initialization. The 30 Hz output was resampled to 50 Hz using linear interpolation for positions and SLERP for quaternions, with velocities recovered by finite differences. When a new plan arrived, the system cross-faded from the old trajectory to the new one over 8 frames (160 ms). The replanning interval adapted to the locomotion mode (for example, every tick for running and less frequently for walking), and mode or heading changes triggered immediate replanning.

Motion streaming. External systems could stream motion data to the robot over ZMQ or ROS2. Incoming frames were merged into a sliding-window `MotionSequence` using a stream merger that handled variable-rate arrival and catch-up. The streamed motion could contain joint-level data, SMPL body poses, VR tracking targets, or any combination thereof. A special token-only streaming protocol allowed an external process to bypass the onboard encoder entirely, sending pre-computed latent tokens that were injected directly into the policy's observation vector. When streaming was active, the frame counter held at the latest available frame,

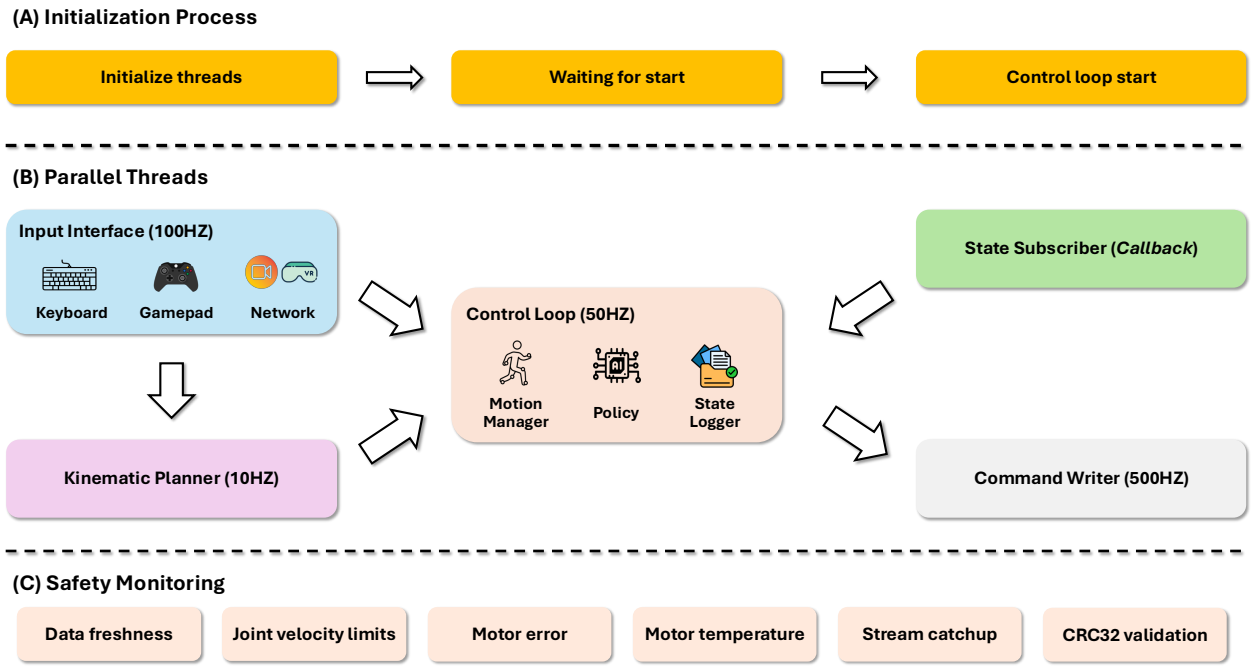

Figure S1: **Deployment architecture of** `SONIC`**.** All inference and management run onboard the Jetson Orin GPU. **(A)** Initialization: worker threads are spawned, the system waits for the operator's start signal, and the control loop begins. **(B)** Parallel threads run concurrently at matched rates and communicate under a latest-data-wins convention: an input interface (100 Hz; keyboard, gamepad, VR, or network), a kinematic planner (10 Hz), the control loop (50 Hz; motion manager, policy, and state logger), a state subscriber callback, and a command writer (500 Hz). **(C)** Safety monitoring continuously checks data freshness, joint-velocity limits, motor error, motor temperature, stream catch-up, and CRC32 validation.

waiting for new data to arrive.

Code is available at https://github.com/NVlabs/GR00T-WholeBodyControl.

S1.3. Kinematic Planner Details

This section provides the full per-mode keyframe specification for the generative kinematic motion planner (the Generative Kinematic Motion Planner section).

Navigation control. Target keyframes were generated by placing a randomly selected segment from the navigation clips of the desired style at the target root trajectory. Despite this simplicity, our model consistently produced natural and smooth motions that aligned well with the specified style. We attributed this to the model's flexible, variable-length motion generation and its robust in-betweening capabilities. Additionally, the autoregressive replanning ensured that the generated motion was continually refreshed before reaching the end of any given clip, thus minimizing dependence on the specific spatial details of the chosen target keyframes. This approach generalized to other motion styles such as walking, running, and crawling.

Entertainment tasks. For entertainment tasks such as boxing, target keyframes were determined by selecting the most expressive segment (such as the frames with maximal arm extension for a punch) from motion clips that matched the desired style. We also supported motion layering, where the upper body was specified and the lower body was generated accordingly by the planner, enabling predefined behaviors.

Interactive manipulation modes. For interactive modes needed in manipulation tasks, such as squatting or kneeling, keyframes were retrieved online from the motion clip library according to the desired height. Unlike traditional approaches that require an extensive motion library, our system needed only a single clip to generate

the full distribution of transitional motions for a given skill.

S1.4. VR Teleoperation and VLA Integration Details

This section provides full details for the teleoperation interfaces (the VR-Based Teleoperation section) and VLA integration (the Foundation-Model-Driven Loco-manipulation section) summarized in the main text.

VR-Based Whole-Body Teleoperation. We developed a full-body VR teleoperation system using the PICO whole-body motion-tracking interface [51]. This required wearing the PICO headset, two ankle trackers, and the handheld VR controllers. The PICO interface provided human motion as full-body human pose estimates in SMPL [52] format. Specifically, the VR interface sent 63-dimensional (21×3) whole-body joint positions, a target root orientation as a 4-dimensional quaternion, 6-dimensional (2×3) wrist joint angles, and 14-dimensional (2×7) dexterous hand joint positions, yielding an 87-dimensional continuous stream per frame. The tracked human motion was streamed in real time to the universal control policy and encoded via the human motion encoder $\boldsymbol{\mathcal{E}}_h$ into the universal token space.

VR-Based 3-Point Teleoperation. To enable scalable and portable data collection for tasks that do not require precise foot placement, we introduced a lightweight mobile bimanual VR teleoperation interface that operated with the PICO headset and two handheld controllers (no ankle trackers needed). The teleoperation interface output a compact command consisting of three upper-body SE(3) poses (head and both wrists), hand joint angles, waist height, a locomotion mode (slow walk or fast walk), and a navigation command specifying the desired root velocity and heading. These signals were fed into the kinematic motion planner and hybrid encoder $\boldsymbol{\mathcal{E}}_m$. The generated motion was then tracked by the universal tracking policy.

3-Point VLA Integration. Using the 3-point interface, we collected 300 teleoperated demonstrations of a mobile pick-and-place task: the robot walked to a randomly placed apple on a table, grasped it with the right hand, and placed it on a randomly positioned plate. We fine-tuned a GR00T N1.5 model [42, 53] on these trajectories. The VLA output the same teleoperation-format control signals (three upper-body poses, base height, navigation command), which were fed into the kinematic planner and hybrid encoder and executed via the universal control policy. On this task, the system attained a 90% success rate over 20 trials.

Whole-Body VLA Integration. For the whole-body tasks, the VLA predicted a 78-dimensional action comprising a 64-dimensional universal motion token and 14-dimensional hand joint angles. Task descriptions and success rates are provided in the main text (the Foundation-Model-Driven Loco-manipulation section, Tab. 1). Additional training details: the soda-can-to-trash-can task used both a multi-object dataset (1,000 trajectories including soda can, paper cup, cardboard, and other disposable items) and a single-object dataset (150 soda-can trajectories). The action space ablation (Tab. 1(B)) compared FSQ tokens against explicit SMPL poses; we observed that SMPL predictions produced unsafe, jerky motions with poor directional control, whereas token predictions yielded substantially smoother behavior and higher task success rate.

S2. Supplementary Discussion

S2.1. Motion Dataset Samples

Fig. S2 shows random samples from our large-scale humanoid motion dataset, illustrating the diversity of behaviors used for training (see the Humanoid Motion Dataset section in the main text).

S2.2. Qualitative Analysis of Success and Failure Motions

To complement the quantitative evaluation in the Motion Tracking section, we visualized out-of-distribution motion sequences from the test-content split to illustrate the range of unseen behaviors our tracker could handle and where it failed (Fig. S3). The successfully tracked OOD motions—imitated despite never being encountered during training—include hip-hop dance, stage bow, sword lunge, and roundhouse kick. The failure cases include zombie crawl (an extreme floor-level motion that exceeds the humanoid's kinematic limits)

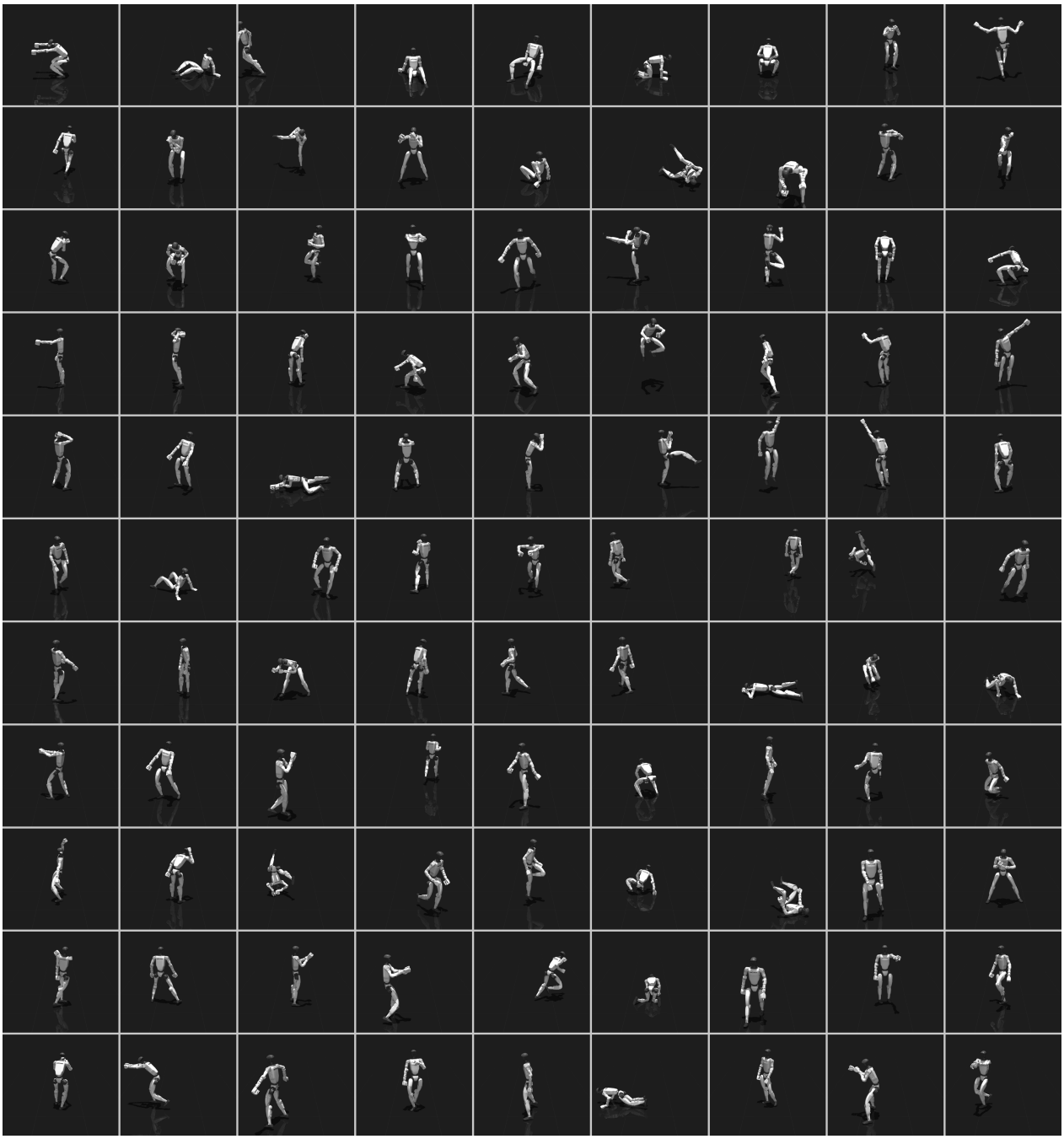

Figure S2: **Random samples from our large-scale humanoid motion dataset.** Each tile shows a frame from a distinct motion sequence retargeted to the Unitree G1 humanoid, illustrating the diversity of behaviors—including locomotion, dance, sports, combat, gestures, and object interactions—that span the 33 motion categories used for training.

and cross-legged sit (a static pose requiring sustained ground contact in a configuration far from the training distribution). These failures indicate that motions requiring sustained or complex ground contact remain challenging for the current system.

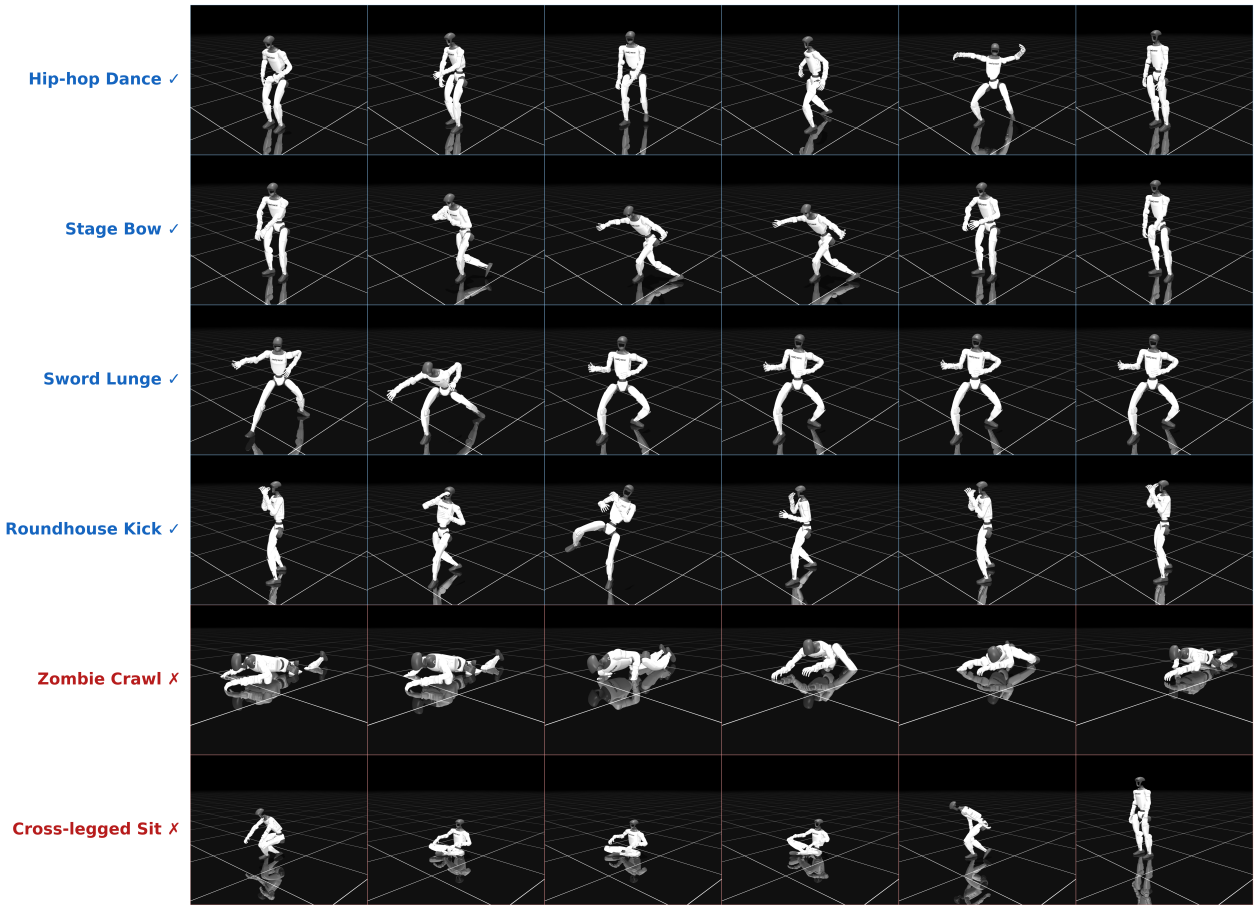

Figure S3: **Out-of-distribution motion sequences from the test-content split.** Unseen motions that are successfully tracked (hip-hop dance, stage bow, sword lunge, roundhouse kick) and unseen motions where tracking fails (zombie crawl, cross-legged sit).

S2.3. Real-World Evaluation Motions

We evaluated SONIC on 124 diverse motion sequences deployed on the real Unitree G1 robot. Fig. S4 shows representative examples from the evaluation set, spanning hip-hop dance, stage bow, high jump, kick, crouch walk, and grovel. All motions shown were successfully tracked in the real world.

S2.4. Robustness to External Pushes

To assess robustness against strong real-world disturbances, we dropped an approximately 11 kg (25 lb) object onto the robot from above head height while it executed the tracking policy (Fig. S5). The robot absorbed the impact, maintained balance, and continued motion tracking without any recovery module or policy adaptation.

S2.5. Scaling Analysis of Specialist Controllers

To support the argument that motion tracking exhibits favorable scaling properties compared to specialist controllers (the Motion Tracking section), we studied the scaling behavior of OpenHomie [13], a state-of-the-art locomotion controller optimized for velocity tracking. Fig. S6 shows OpenHomie's velocity tracking error and survival rate as a function of compute scale, ranging from 1 GPU (4k environments) to 32 GPUs (4 nodes). OpenHomie's performance peaked at 8 GPUs (0.18 m/s tracking error, 95.0% survival) and did not improve

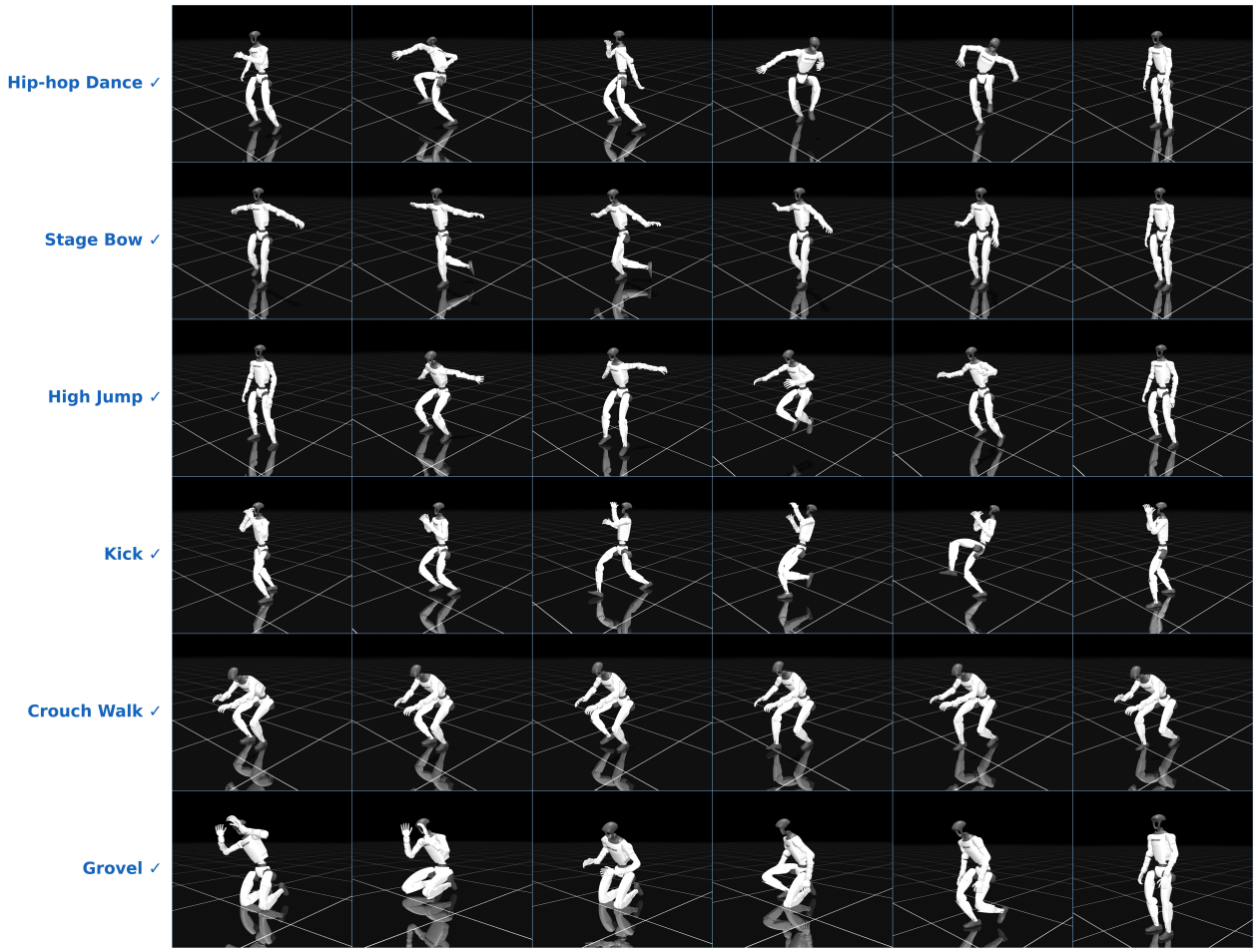

Figure S4: **Representative motions from the 124-sequence real-world evaluation set.** All shown motions are successfully tracked on the physical Unitree G1 robot.

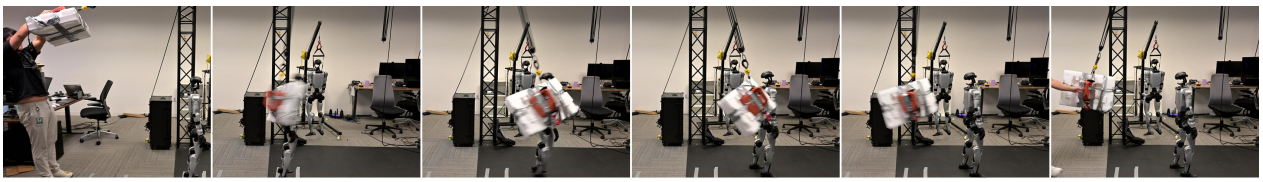

Figure S5: **Robustness test.** An approximately 11 kg (25 lb) object was dropped onto the robot from above head height during policy execution. The robot absorbed the impact, maintained balance, and continued tracking. No recovery module or policy adaptation was used.

when scaling to 32 GPUs (0.29 m/s error, 91.2% survival). This stood in contrast to SONIC, where scaling from 2 to 16 nodes yielded consistent improvements in both success rate and tracking accuracy (Fig. 2A–C). We attributed this difference to the nature of the training objective: OpenHomie used task-specific reward engineering for locomotion, which saturated once the policy mastered the target behavior, whereas motion tracking provided dense per-frame supervision across a diverse motion distribution that continued to benefit from additional capacity and data throughput.

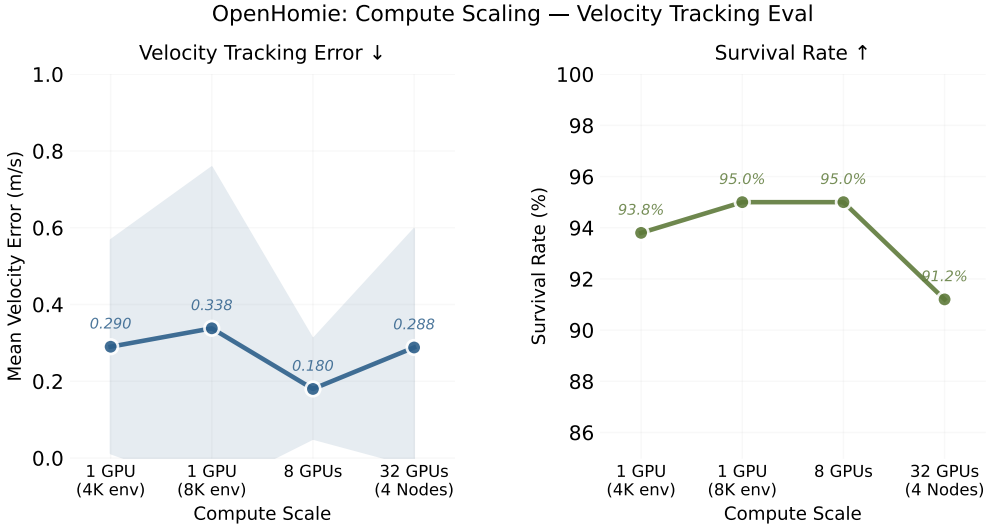

Figure S6: **OpenHomie velocity tracking evaluation vs. compute scale.** Mean velocity tracking error (m/s) and survival rate (%), each shown as a function of training compute. Each configuration was evaluated over n=80 runs (20 commanded velocities from 0.1 to 2.0 m/s $\times$ 4 directions, 5 s each). On the tracking-error plot, the line denotes the mean and the shaded band mean $\pm$1 standard deviation (s.d.) over the n=80 runs; the survival rate is the percentage of those runs completed without a fall.

S2.6. Latent Space Alignment of the Multi-Encoder Design

Our multi-encoder design mapped heterogeneous input types (robot motion, human SMPL poses, and hybrid teleop commands) into a shared token space, aligned by the consistency losses $\mathcal{L}_{\text{token}}$ and $\mathcal{L}_{\text{cycle}}$ (the Validation of Key Design Choices section). Fig. S7 visualizes this alignment on a crawling motion, with and without the consistency losses. Removing the consistency losses caused an $8\times$ increase in cross-encoder divergence, confirming that they were necessary for cross-encoder alignment.

S3. Supplementary Videos

The project site hosts a collection of high-resolution videos that complement the results in the main paper, demonstrating large-scale motion tracking, interactive control, multi-modal interfaces, teleoperation, and foundation-model-driven loco-manipulation on a Unitree G1. The videos are provided as Supplementary Movies S1 to S5 and are cited in the main text in order of appearance.

S3.1. Movie S1: Summary Video

These clips demonstrate universal motion tracking across diverse behaviors, together with robustness under strong external perturbations: an operator drops an approximately 11 kg (25 lb) object onto the robot during policy execution, and the robot absorbs the impact, maintains balance, and continues tracking without any recovery module or policy adaptation (see the Robustness to External Pushes section).

S3.2. Movie S2: Interactive Motion Control

These videos demonstrate the real-time kinematic motion planner (the Generative Kinematic Motion Planner section) coupled with the universal tracking policy.

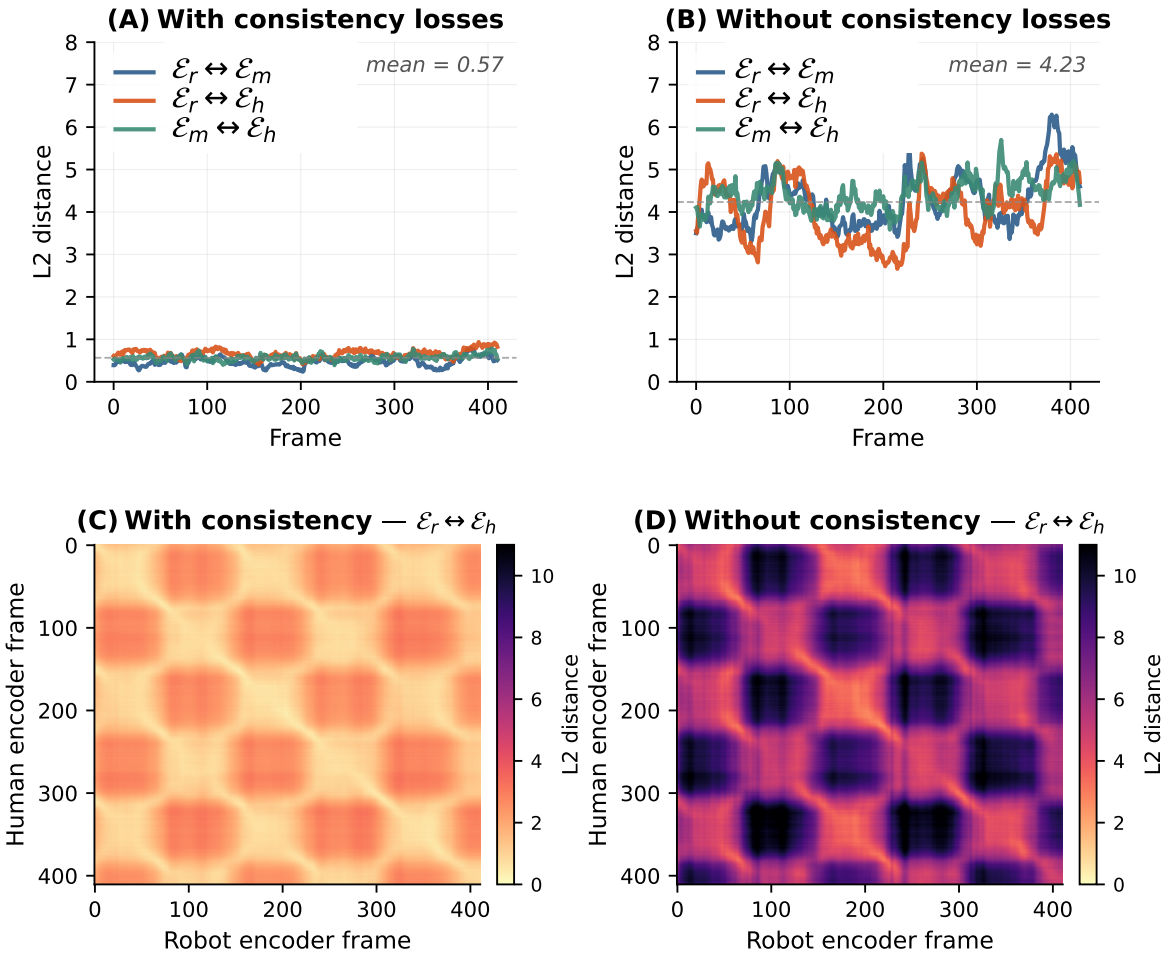

Figure S7: **Consistency losses align the multi-encoder latent space.** Latent space alignment with and without the consistency losses, visualized on a crawling motion. **(A and B)** Diagonal L2 distance between encoder pairs. **(C and D)** Cross-encoder distance matrices. With the consistency losses, matching frames produce aligned tokens; without them, the alignment breaks down.

Stylized Locomotion. Users control the humanoid through continuous velocity and heading commands via gamepad. The planner replans at 10 Hz, producing responsive and natural movements. Clips show smooth transitions between motion styles: happy walk, running, stealth walk, injured walk, and sprinting.

Interactive Boxing. SONIC executes jabs, hooks, stance shifts, and reactive upper-body movements as continuous, unbroken sequences driven by the kinematic planner.

Squatting, Kneeling, and Crawling. Videos show arbitrary pelvis-height control (0.3–0.8 m), kneeling variations, elbow crawling, hand-knee crawling, and transitions. These skills enable locomotion in confined spaces and downstream teleoperation behaviors.

S3.3. Movie S3: Video and Multi-Modal Control

Video Teleoperation. Using video as input and GEM [50] for pose estimation at $\geq$60 fps, the humanoid tracks and reproduces complex motions from human demonstrations in real time. Clips include walking, running, turning, jumping, dancing, and floor motions.

Text-Driven Motion. SONIC responds to natural-language commands such as "walk forward", "punch with your left hand", or "act like a monkey" and can track the generated motion from GEM [50].

Music-Conditioned Motion. GEM [50] generates dance motions conditioned on melodic and rhythmic structure; SONIC tracks the generated choreography in real time. Both text and music modalities use the same universal control policy with no retraining.

S3.4. Movie S4: VR Teleoperation

Full-Body VR Control. Using the PICO whole-body motion-tracking interface [51] (headset, ankle trackers, and controllers), full-body SMPL pose streams drive SONIC via the human-motion encoder $\mathcal{E}_h$. The robot performs walking, sidestepping, crouching, and loco-manipulation with low latency.

3-Point VR Teleoperation. A lightweight teleoperation mode using only the headset and handheld controllers (no ankle trackers). The user provides upper-body SE(3) poses via the controllers; the kinematic planner generates the lower body. Videos show mobile manipulation, navigation, reaching, and object interaction tasks.

S3.5. Movie S5: Foundation-Model-Driven Loco-Manipulation

These clips show a GR00T N1.5 VLA model [42, 53] driving SONIC via the universal token interface. Videos cover all five VLA tasks: apple to plate (3-point interface), object pickup (carrot, scrub), open trash can with foot, soda can to trash can (multi-step: pick up, navigate, step on pedal, throw), and drill and box relocation. All policies are fully autonomous with no human intervention during execution.

S4. Supplementary Data

Tabulated source data underlying the quantitative main-text figure panels are provided as a separate machine-readable file, Data S1 (a multi-sheet spreadsheet with one sheet per figure panel).